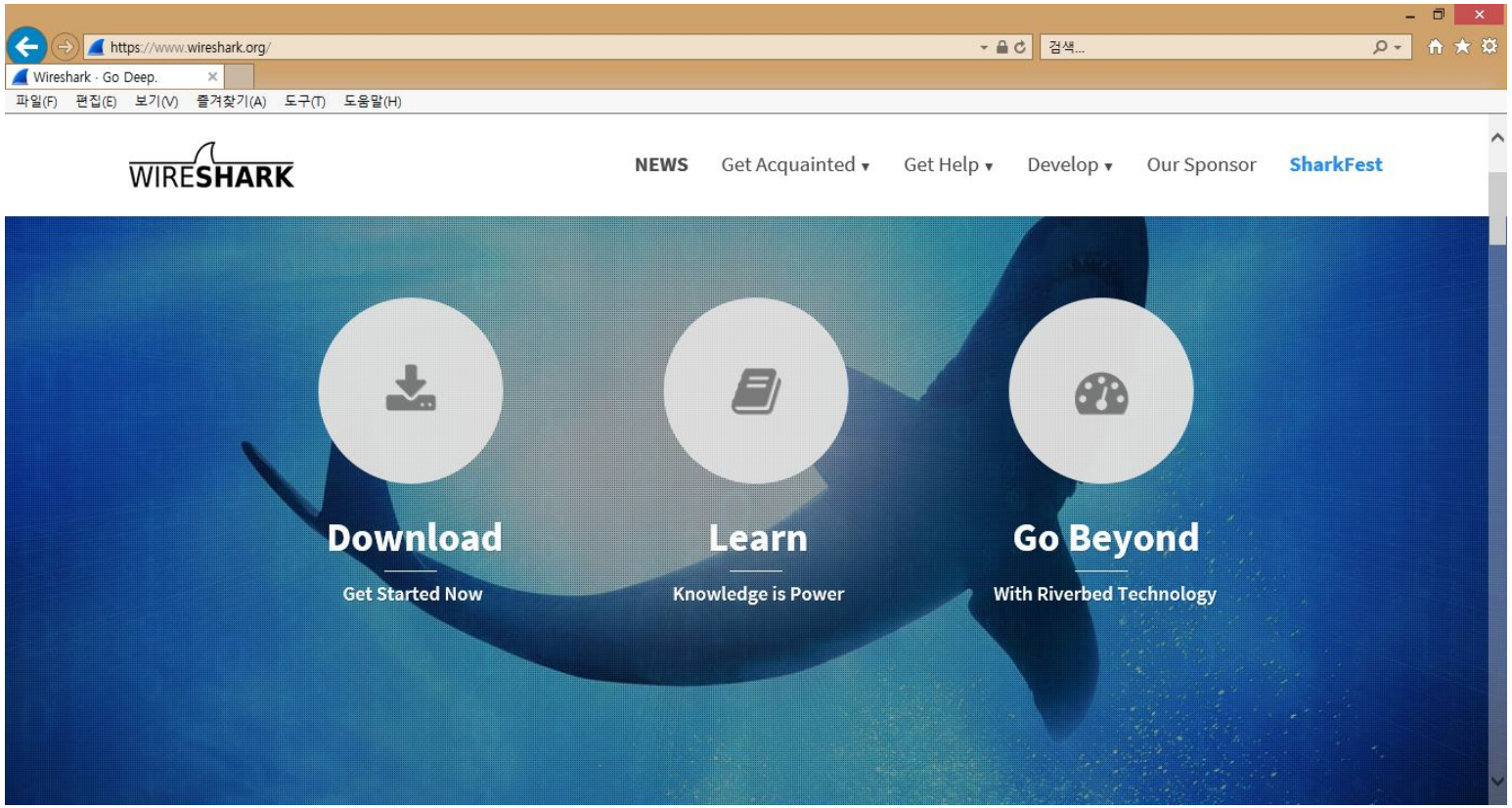
Sniffing 환경구성

- 패킷 스니핑(sniffing) 또는 프로토콜 분석(protocol analysis) 툴
- 네트워크를 통해 전달되는 데이터를 수집하고 해석



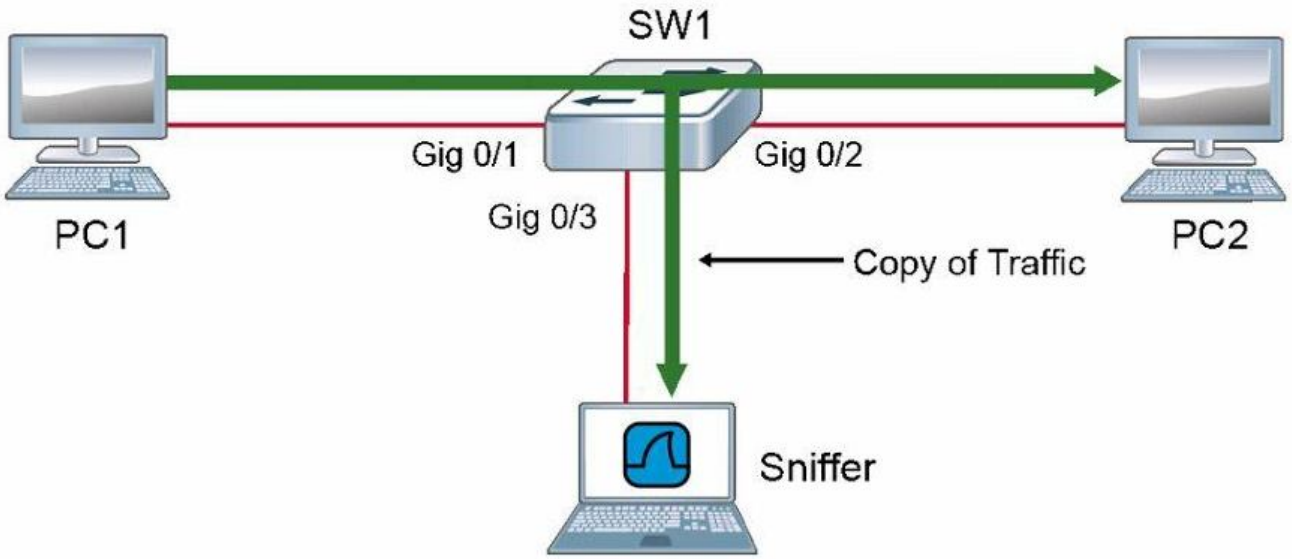
http://www.wireshark.org

* 패킷 수집과 패킷 분석

	LINUX	MS-Window
패킷수집	TCPdump	TShark
패킷분석	Wireshark	Wireshark

무차별모드(Promiscuous mode)

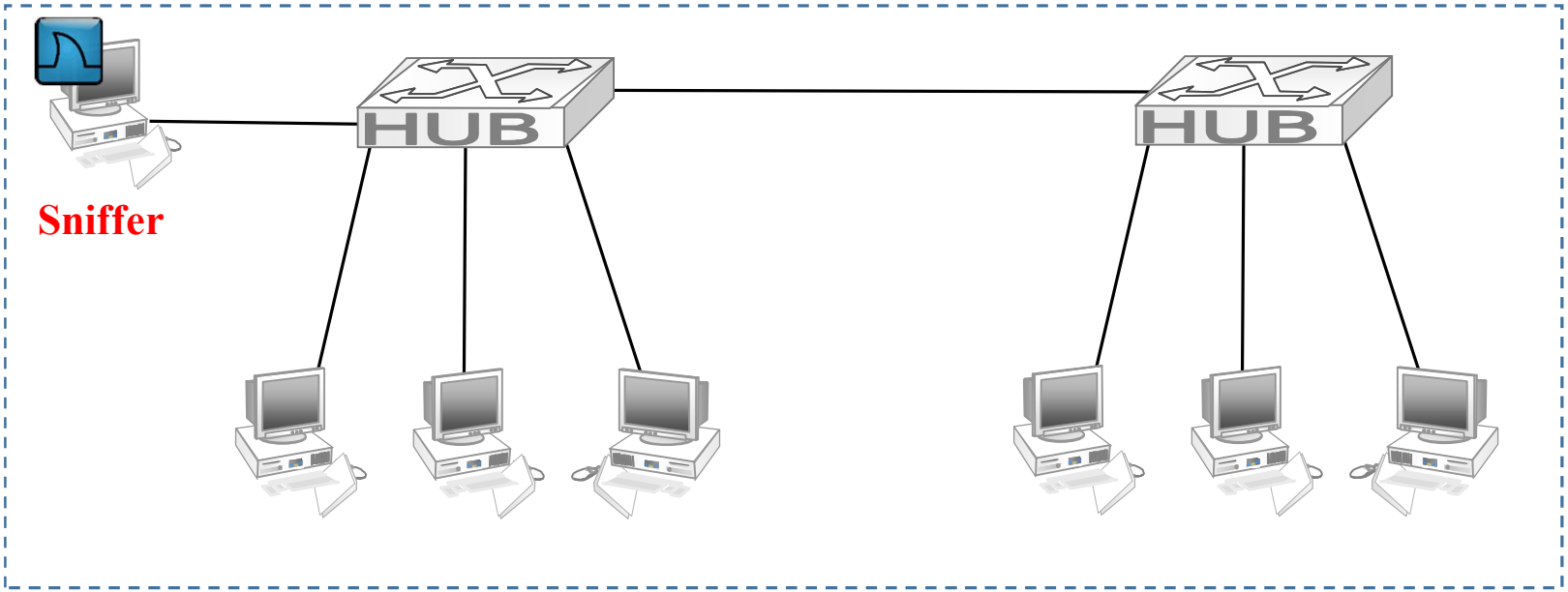
- 데이터 링크 계층과 네트워크 계층의 주소 필터링을 해제한 모드
- 주소 지정에 관계없이 호스트의 프로세서에 모든 패킷을 전달
- 패킷이 CPU에 전달되면 분석을 위해 패킷 스니핑 애플리케이션에 넘겨짐
- 유닉스 또는 리눅스에서는 'ifconfig eth0 promisc'로 설정
- Window에서는 winpcap / npcap설치로 무차별 모드 설정



Packet Sniffing 환경

❶ 허브 환경에서의 스니핑

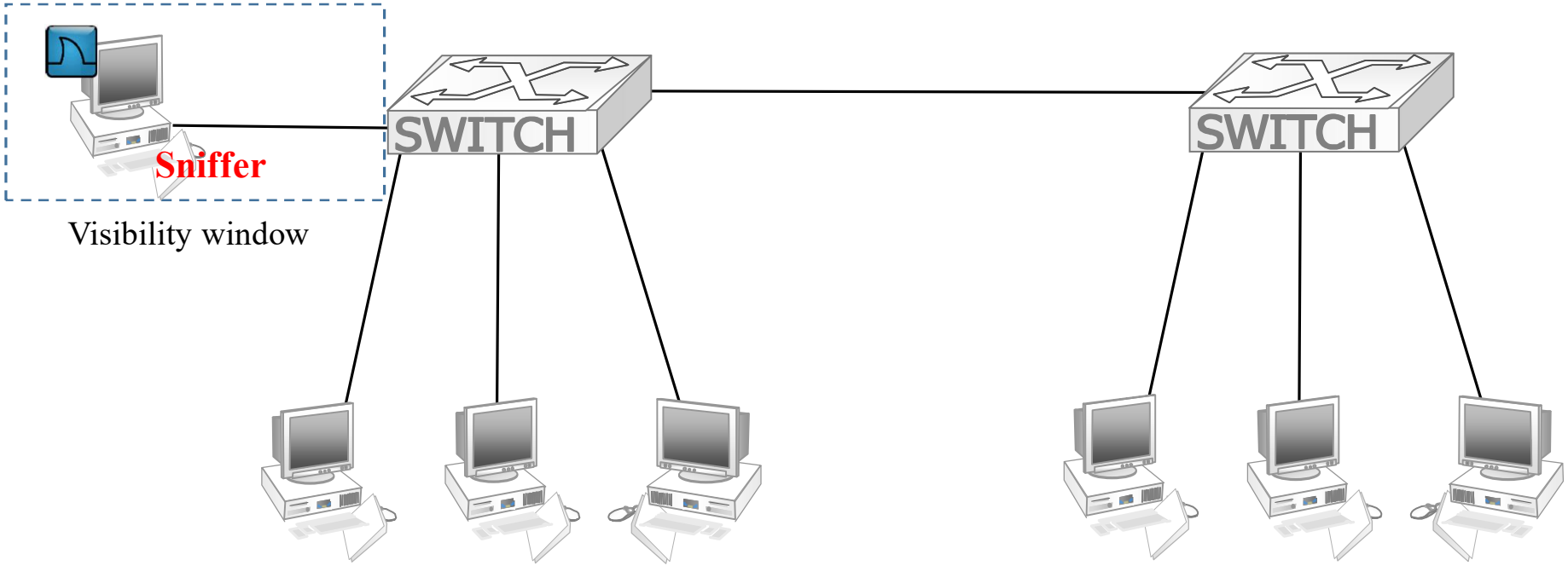
- 허브를 통해 전송되는 트래픽은 해당 허브에 연결된 모드 포트를 통과



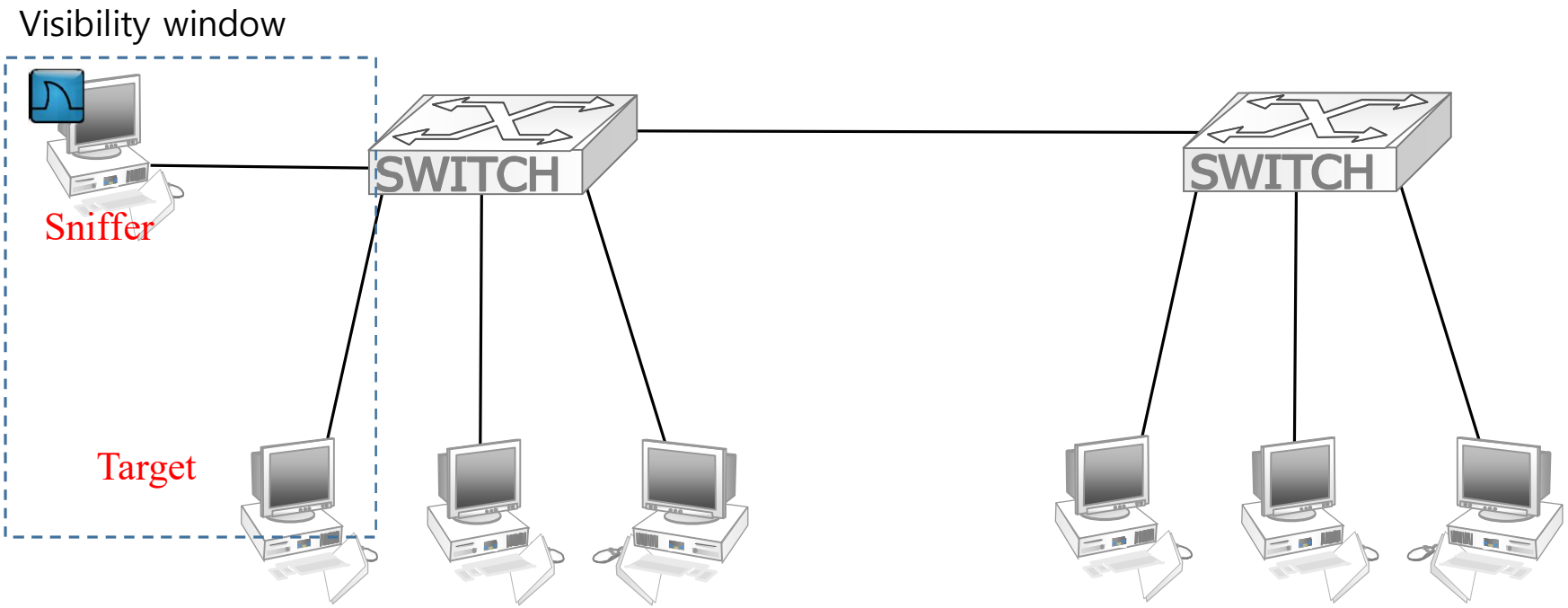
Visibility window (가시성 창)

❷ 스위치 환경에서 스니핑 방법

- 포트 미러링(port mirroring)
- 허빙 아웃(hubbing out)
- 탭 사용 (tapping)



포트 미러링

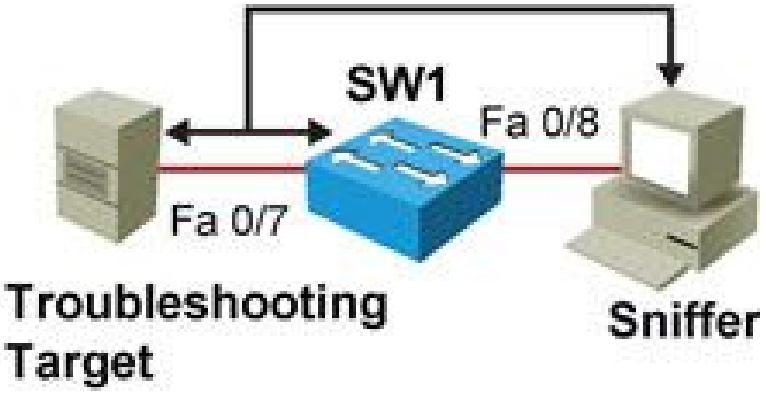


•Switched Port Analyzer (SPAN)

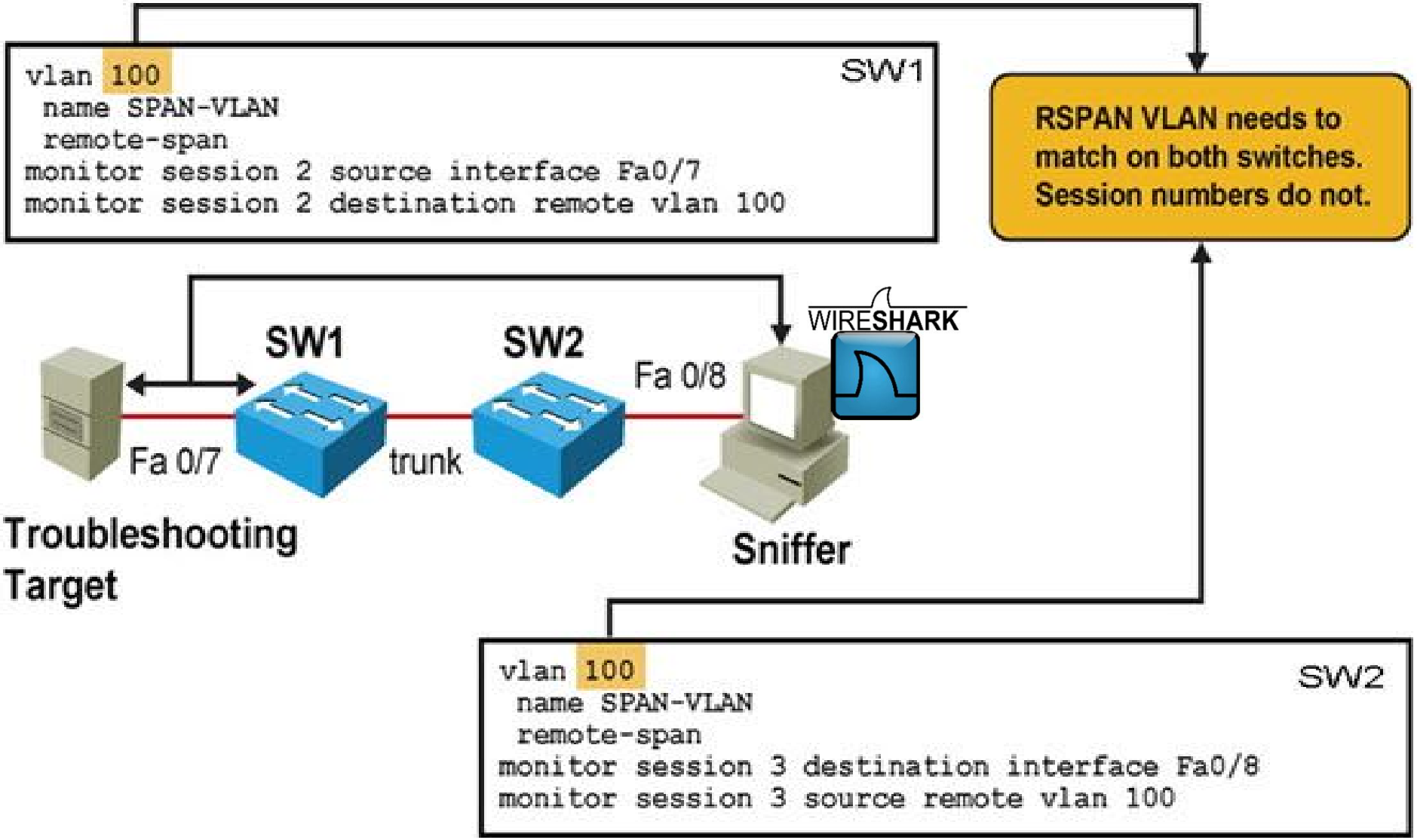
```
monitor session 1 source interface Fa0/7
monitor session 1 destination interface Fa0/8
```

Sources and destinations that form a single SPAN session are identified by a session number

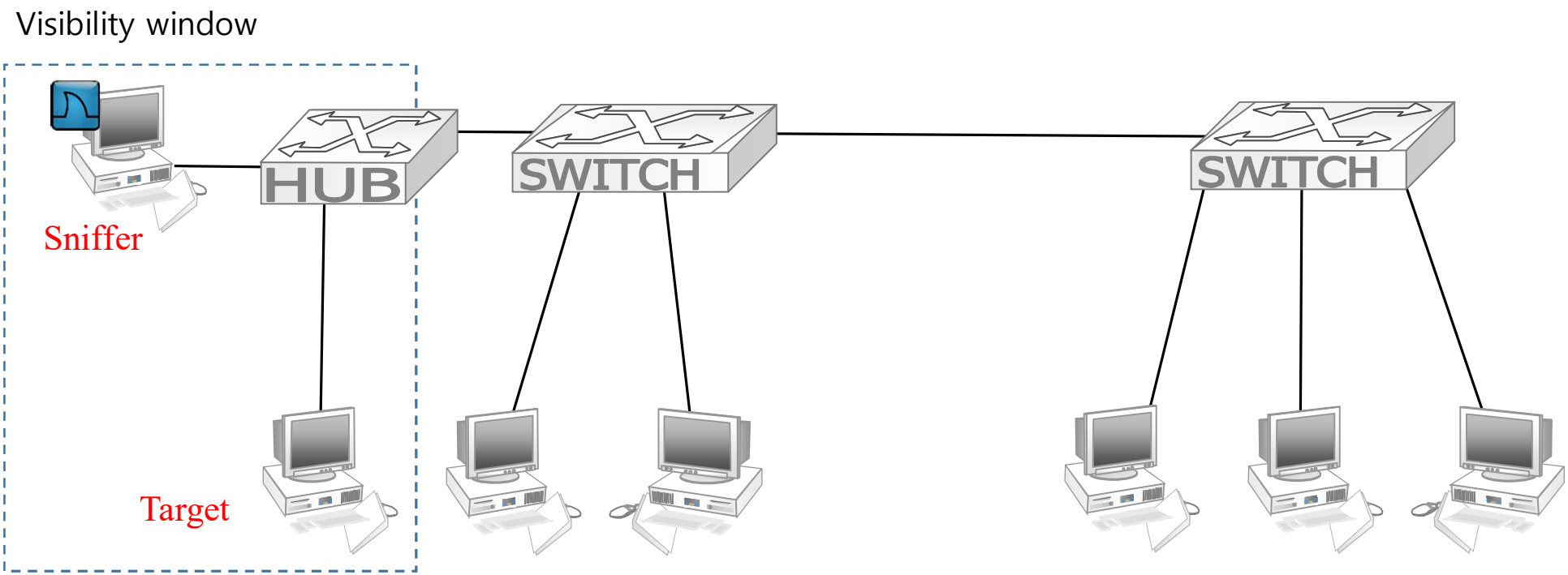
```
SW1#show monitor
Session 1
---------
Type                    : Local Session
Source Ports            :
    Both                : Fa0/7
Destination Ports       : Fa0/8
    Encapsulation       : Native
          Ingress       : Disabled
```



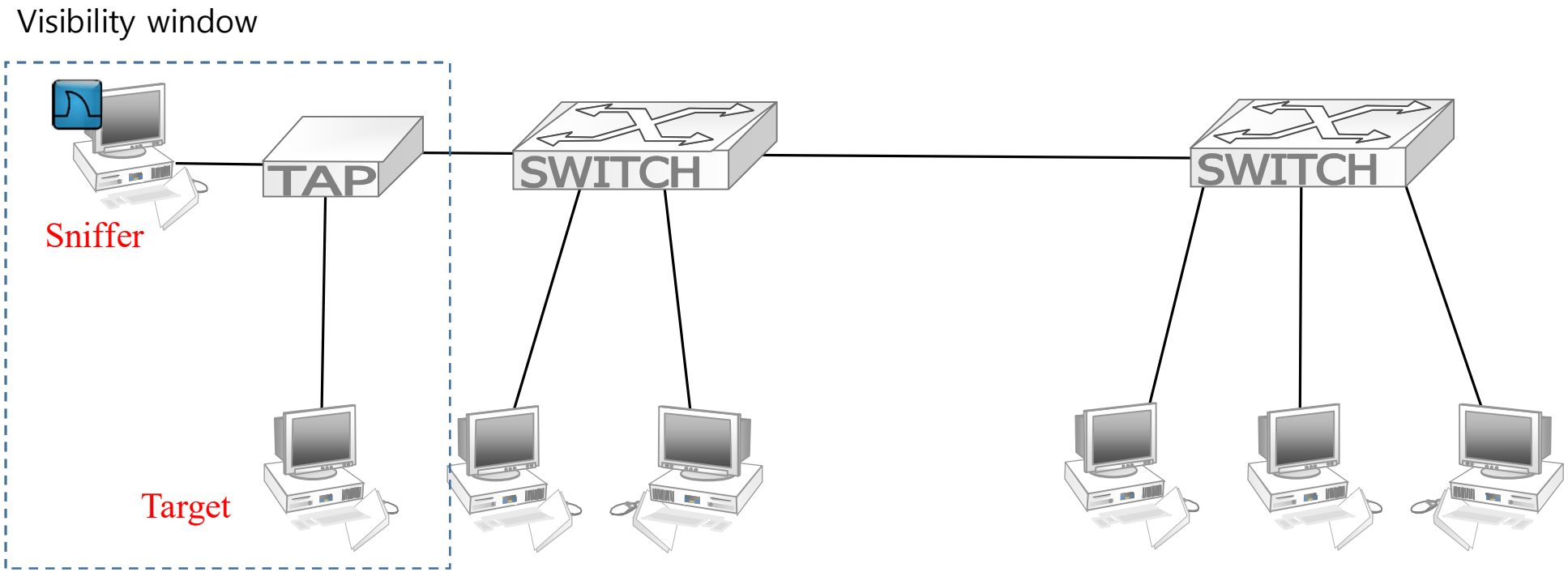
•Remote Switched Port Analyzer (RSPAN)



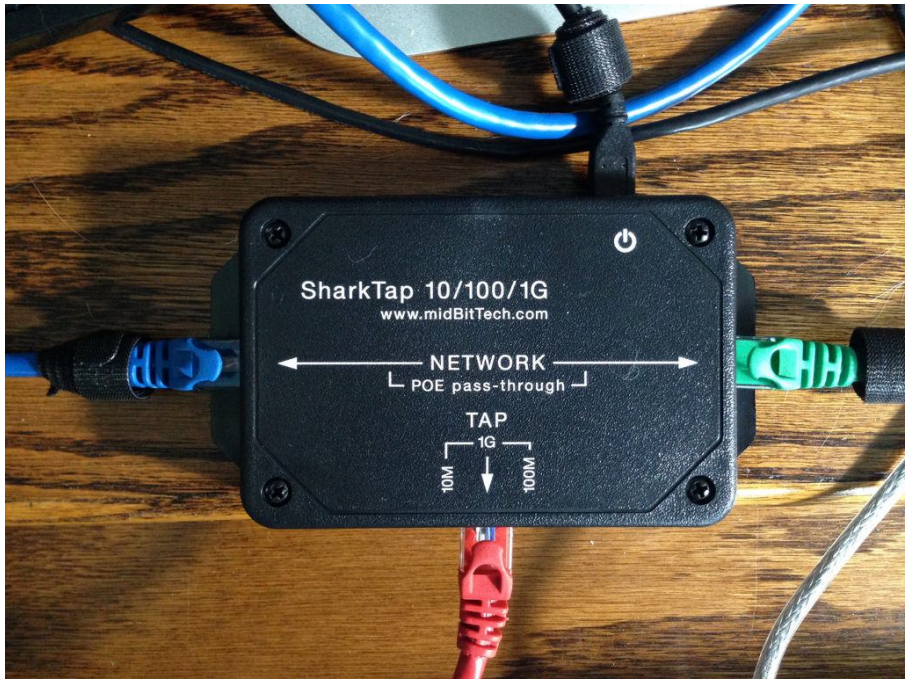
허빙 아웃(hubbing out)



탭 사용 (tapping)

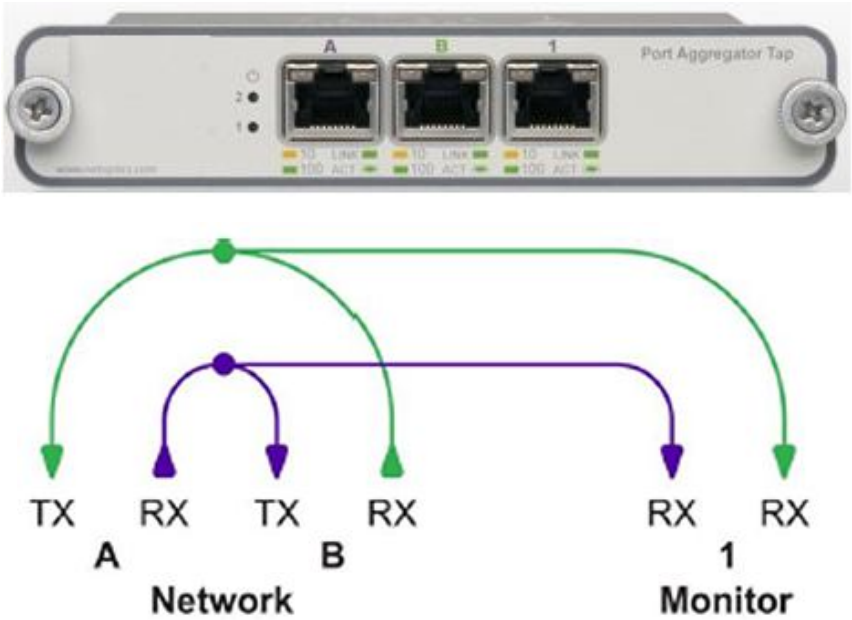


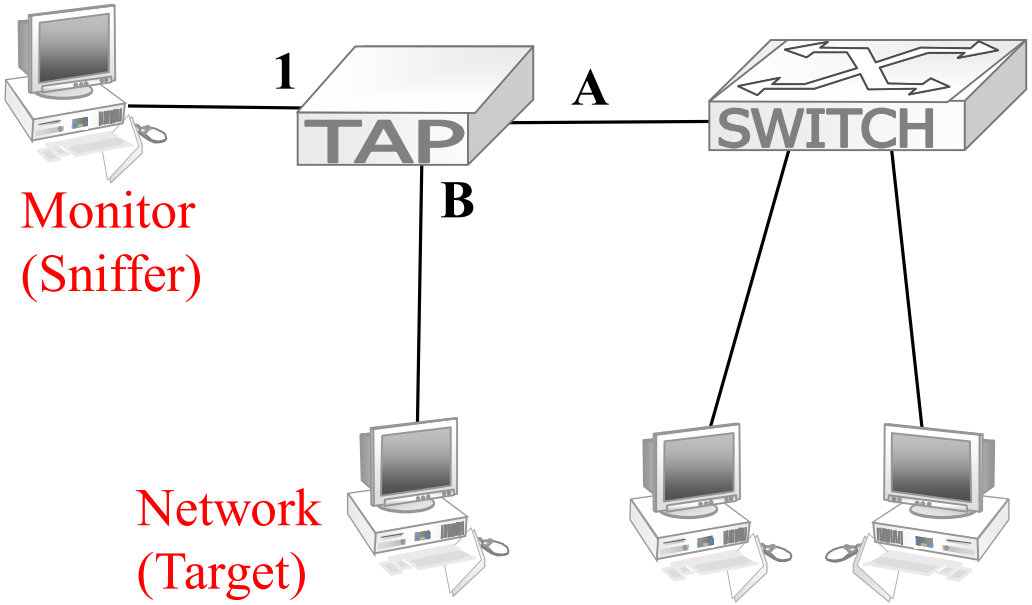
스위치 환경에서의 스니핑 - Tap



스위치 환경에서의 스니핑 - Aggregated Tap

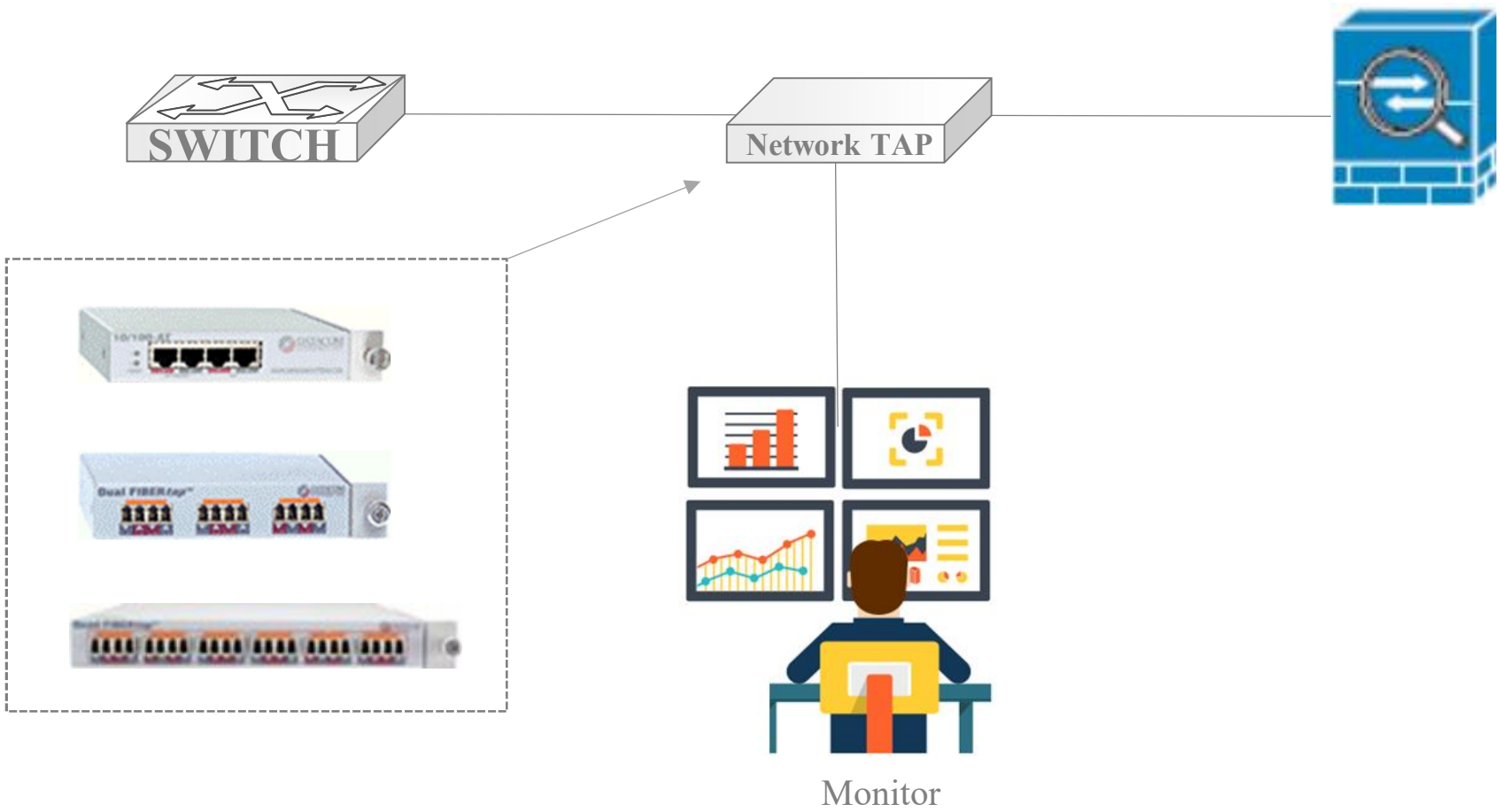
- 3개의 포트로 구성
- 양방향 트래픽을 스니핑하기 위한 물리적 모니터 포트를 하나만 갖고 있음





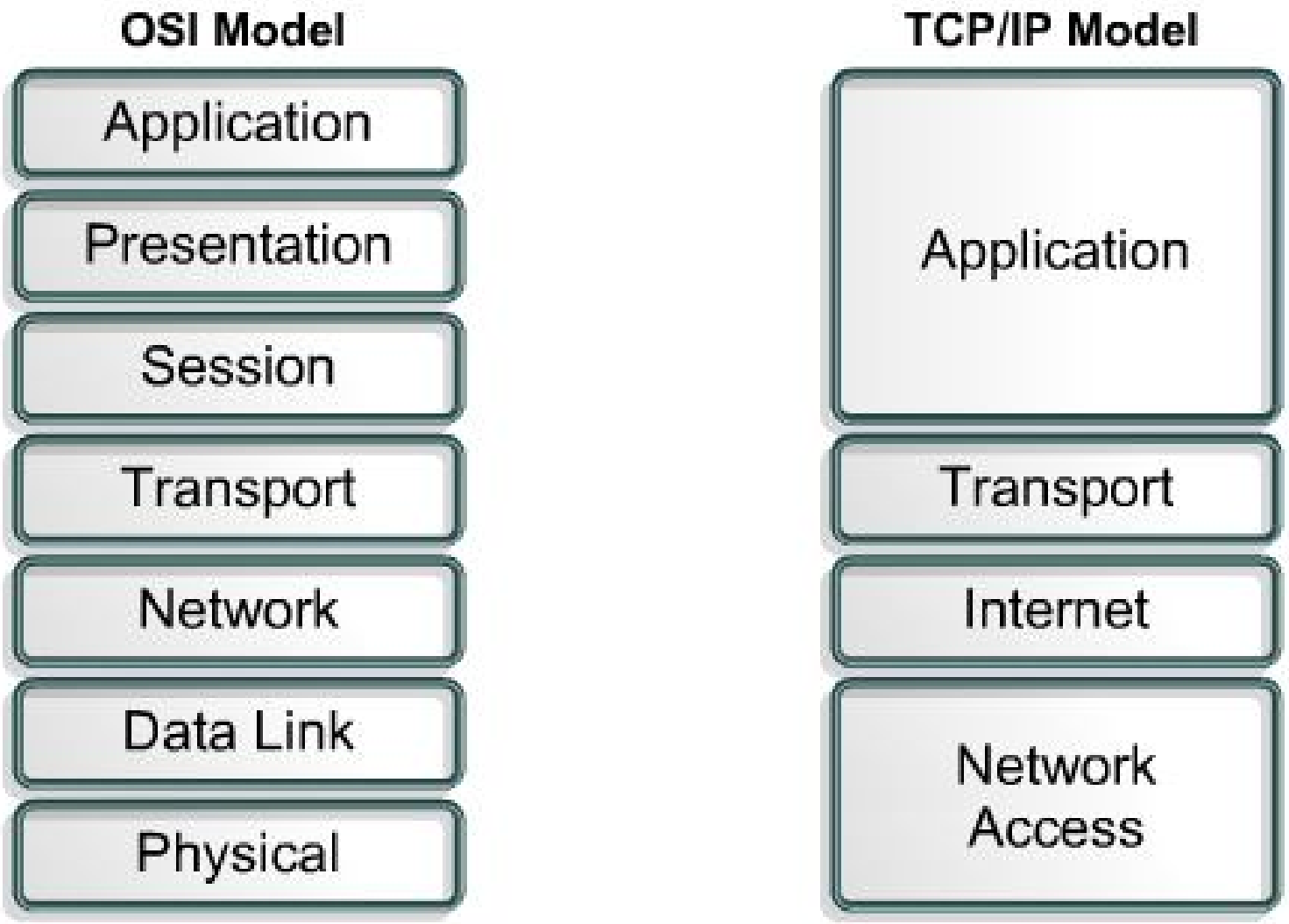
Network TAP

- Network상의 In-Line 한 구간에 이동하는 Packet Data를 복사하여 Monitor 장비로 보내주는 역할을 하는 가장 기본적인 TAP

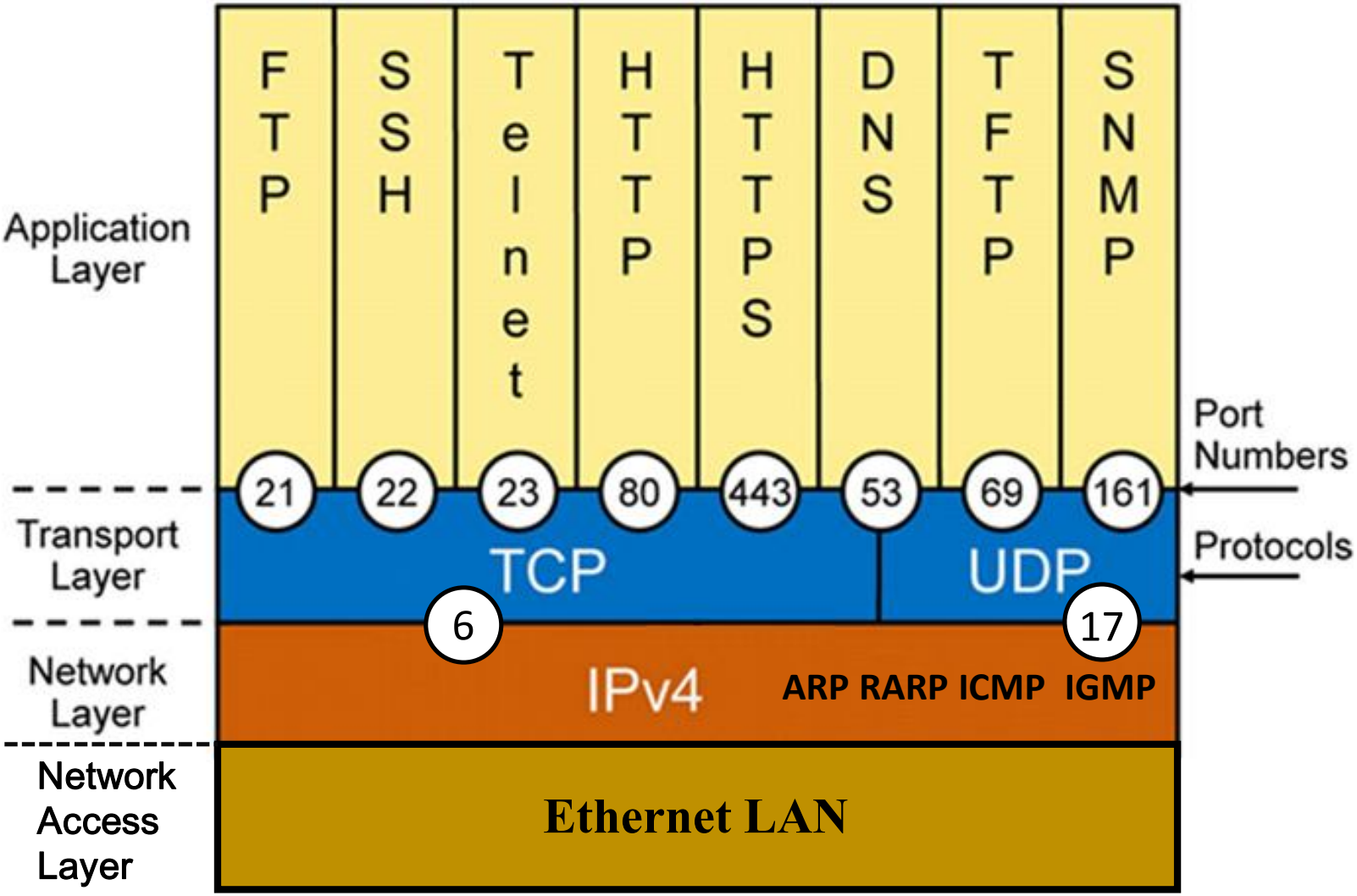


TCP/IP 프로토콜 분석

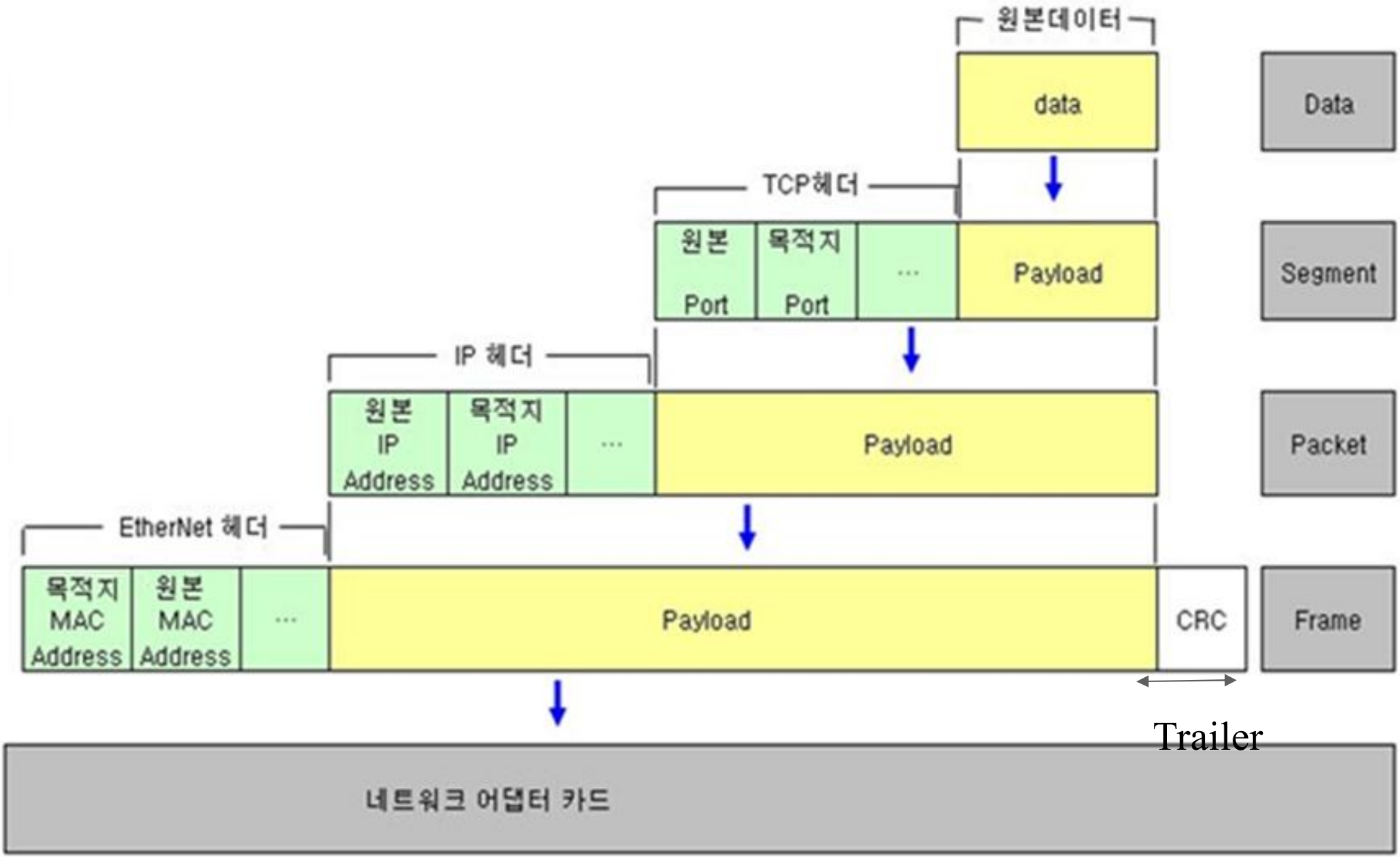
TCP/IP 통신 모델


OSI Model
Application
Presentation
Session
Transport
Network
Data Link
Physical
TCP/IP Model
Application
Transport
Internet
Network Access

TCP/IP Protocol Stack

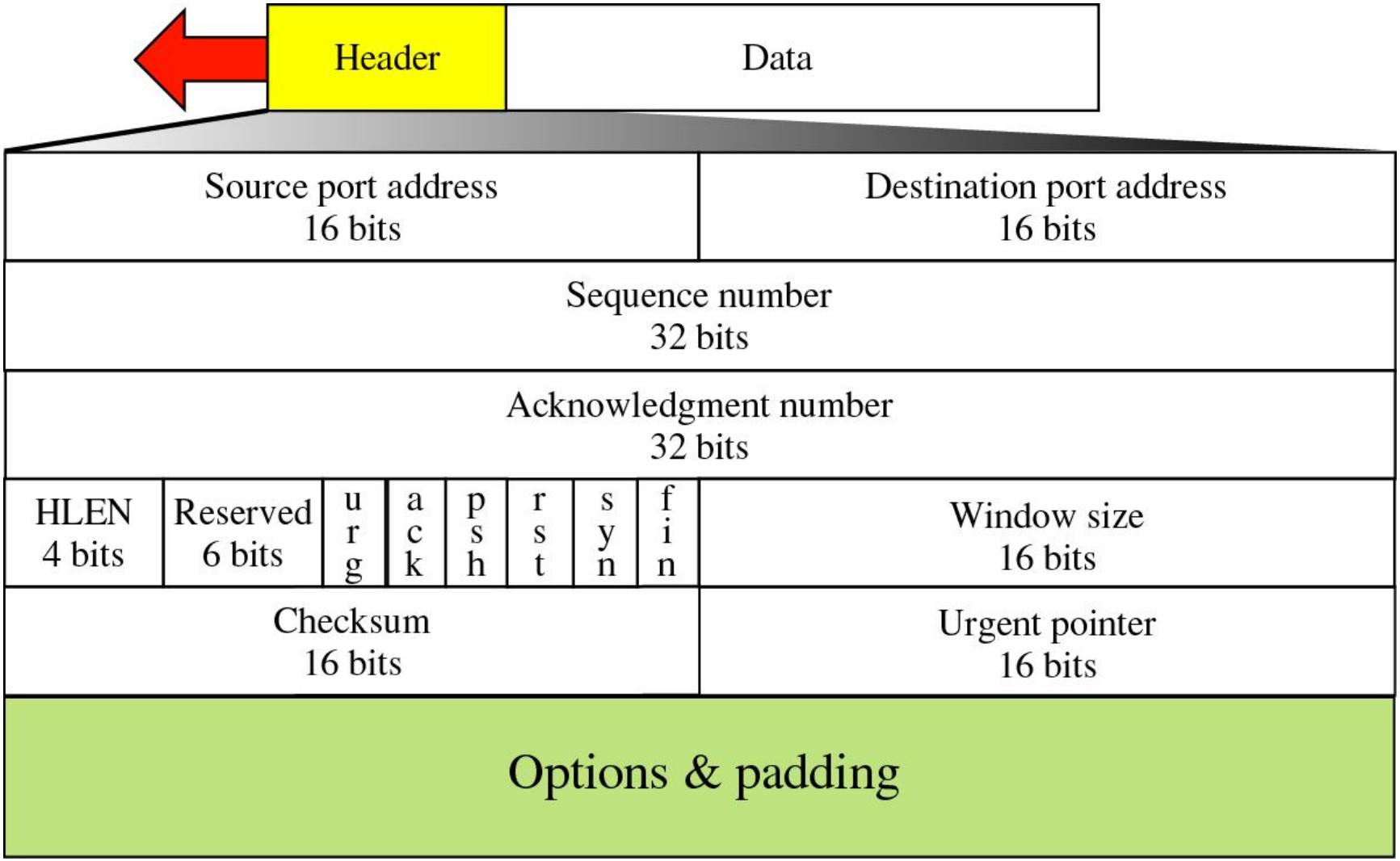


Payload & Encapsulation



1) TCP Header

- TCP 헤더는 기본이 20바이트이며 옵션을 포함한 경우 최대 60바이트로 구성



TCP 연결 관리

❶ TCP 데이터 전송 전

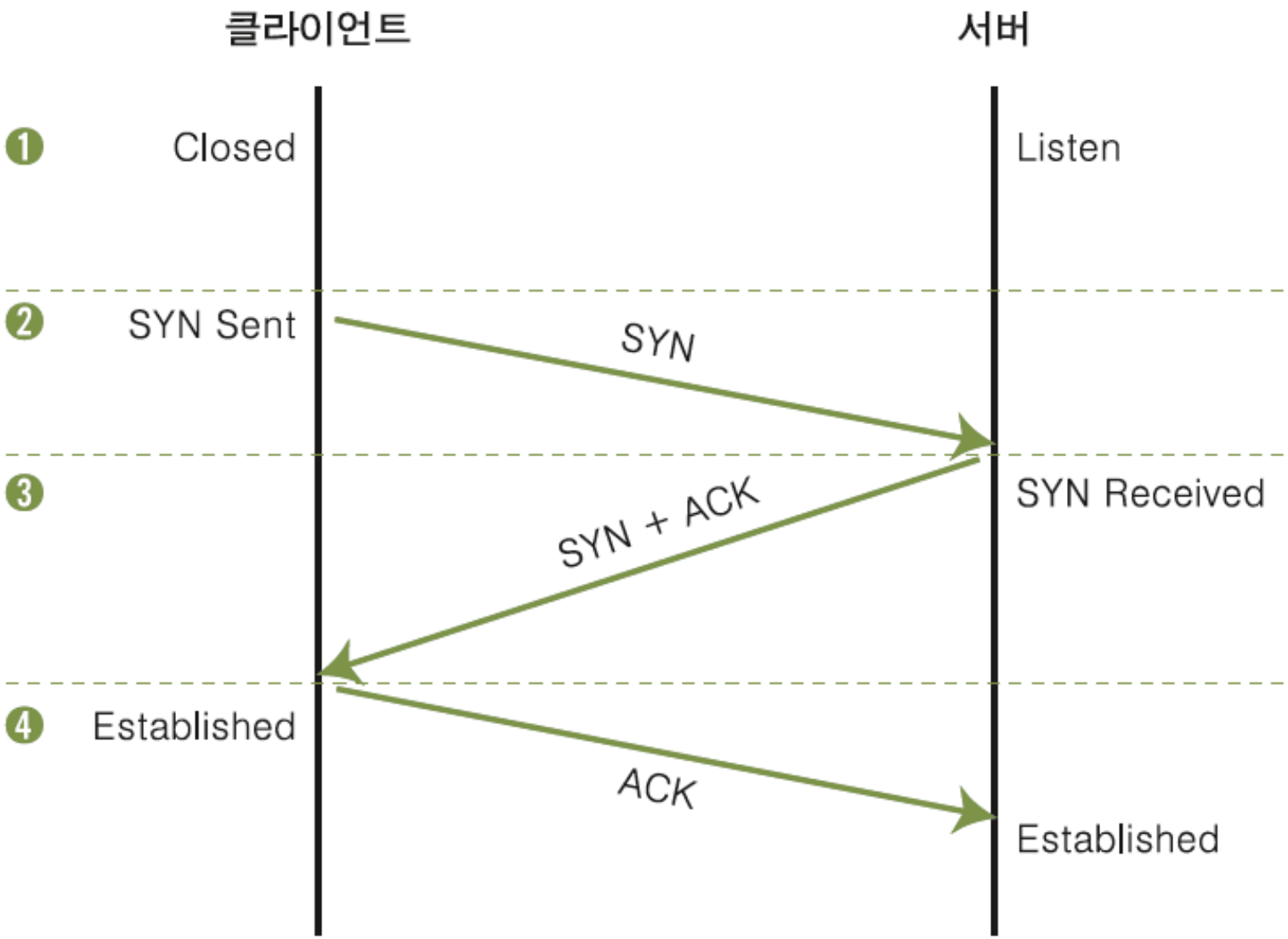
- 3 Way Handshaking

❷ TCP 데이터 전송 과정

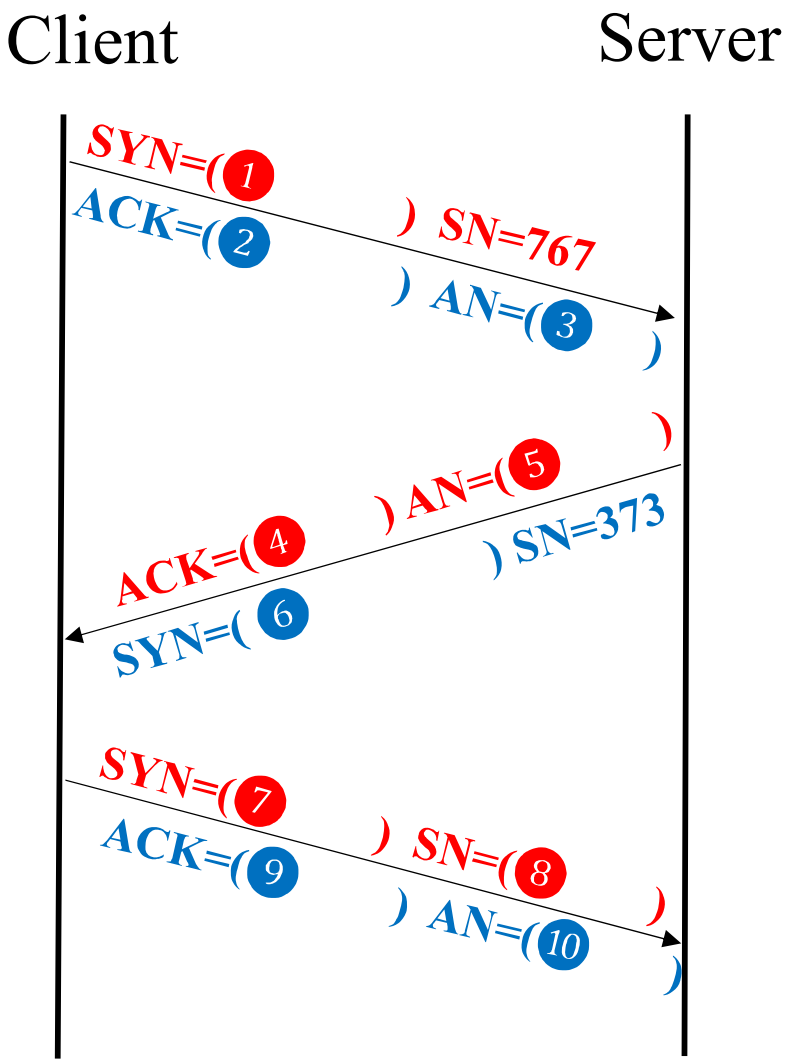
❸ TCP 연결 종료 과정

- 4 Way Handshaking

TCP 연결 설정 (3-Way Handshaking)

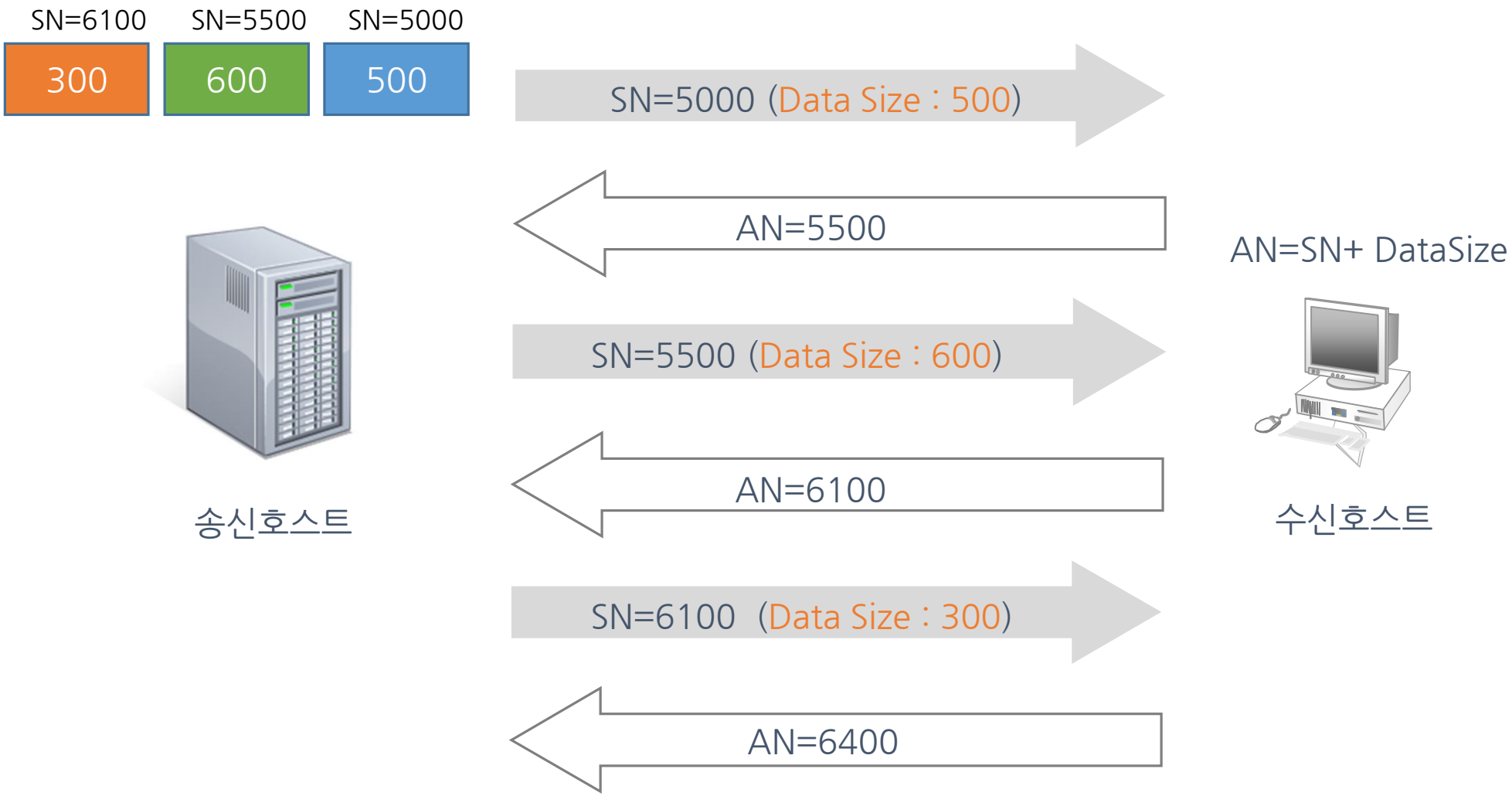


* 3WHS의 일부분이다. 괄호부분을 모두 합한 값은?

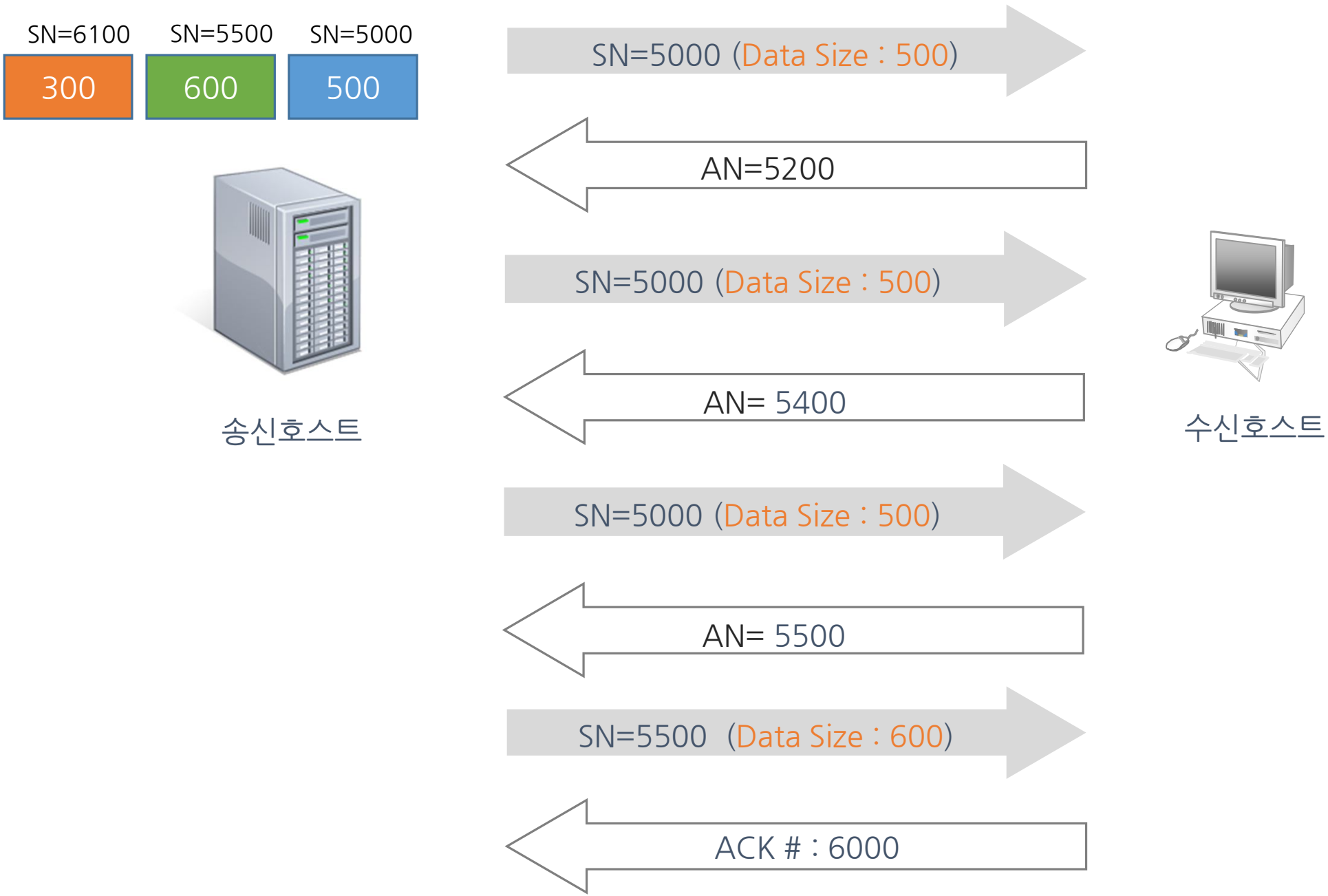


❶ + ❷ + ❸ + ❹ + ❺ + ❻ + ❼ + ❽ + ❾ + ❿

❶ 정상적인 트래픽 전송 과정

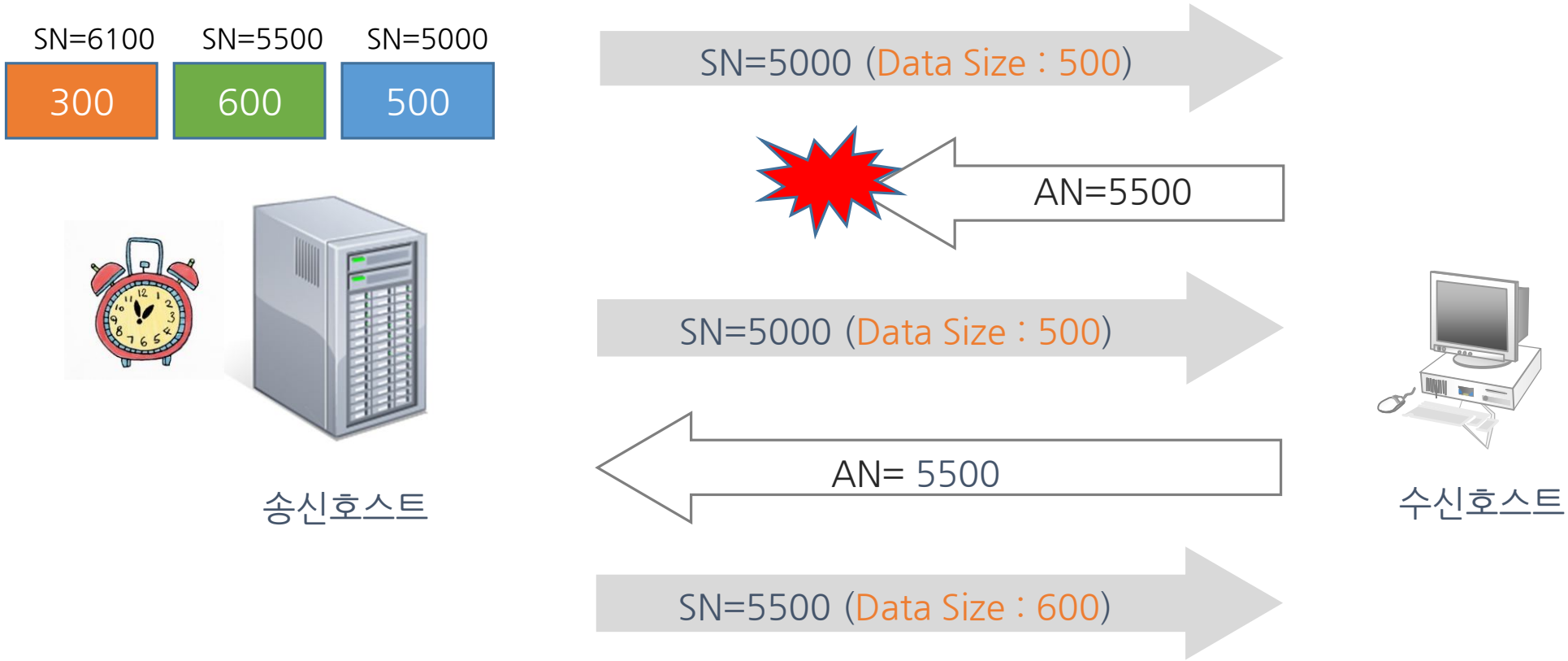


❷ 비정상적인 트래픽 전송 과정

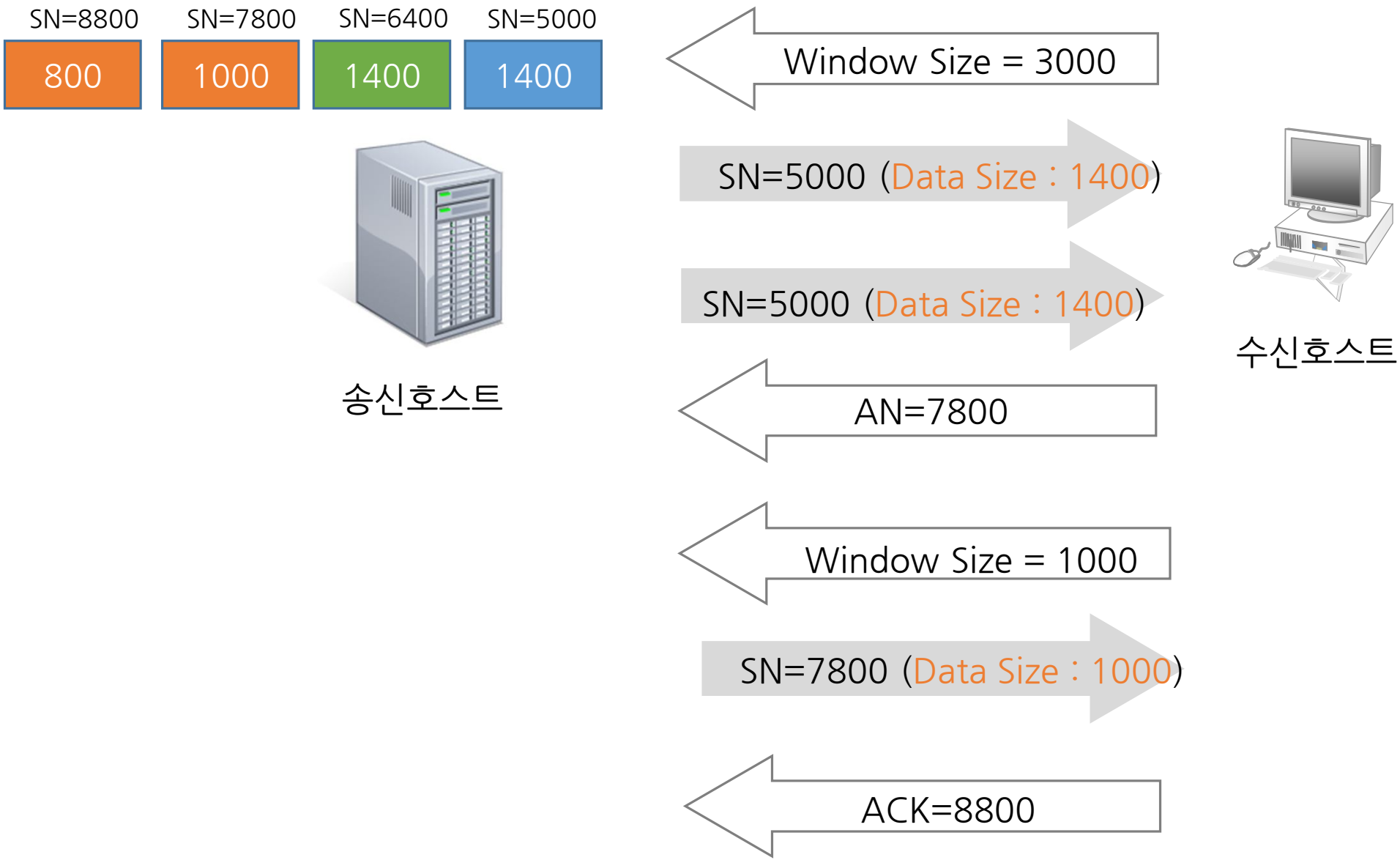


❷ 비정상적인 트래픽 전송 과정

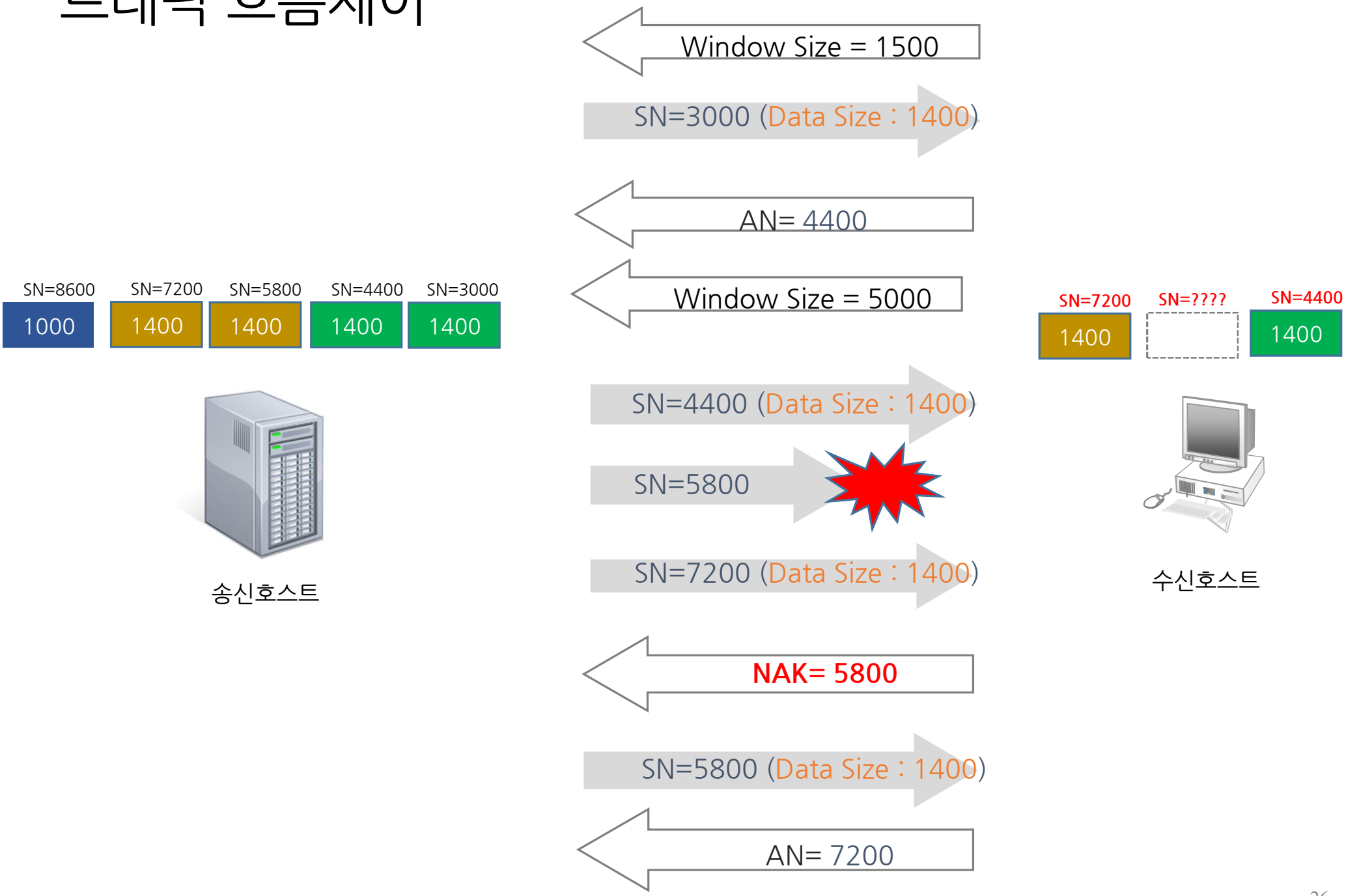
(예) 송신측에서 재전송 시도



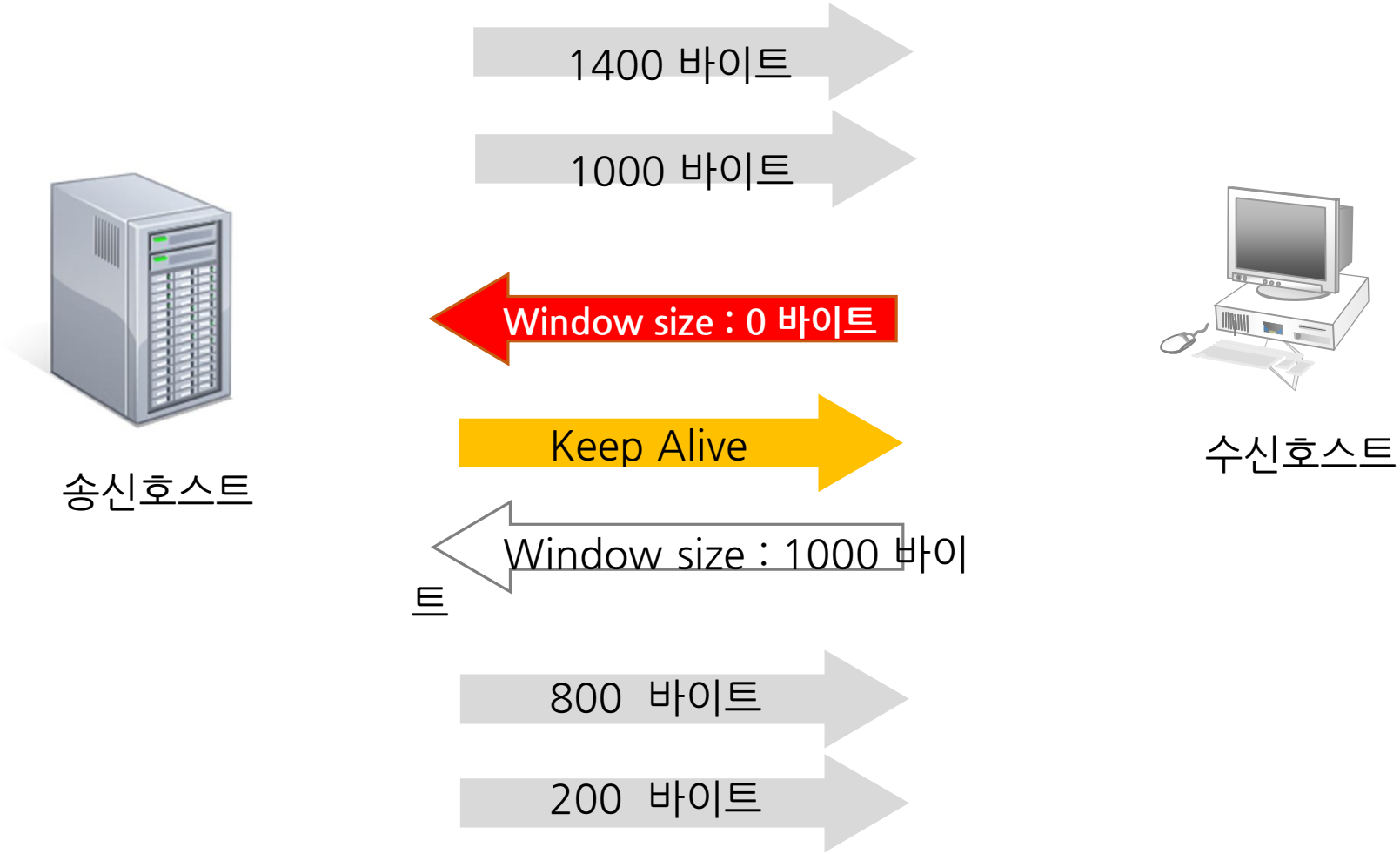
트래픽 흐름제어


SN=8800
SN=7800
SN=6400
SN=5000
800
1000
1400
1400
Window Size = 3000
SN=5000 (Data Size : 1400)
SN=5000 (Data Size : 1400)
AN=7800
Window Size = 1000
SN=7800 (Data Size : 1000)
ACK=8800
송신호스트
수신호스트

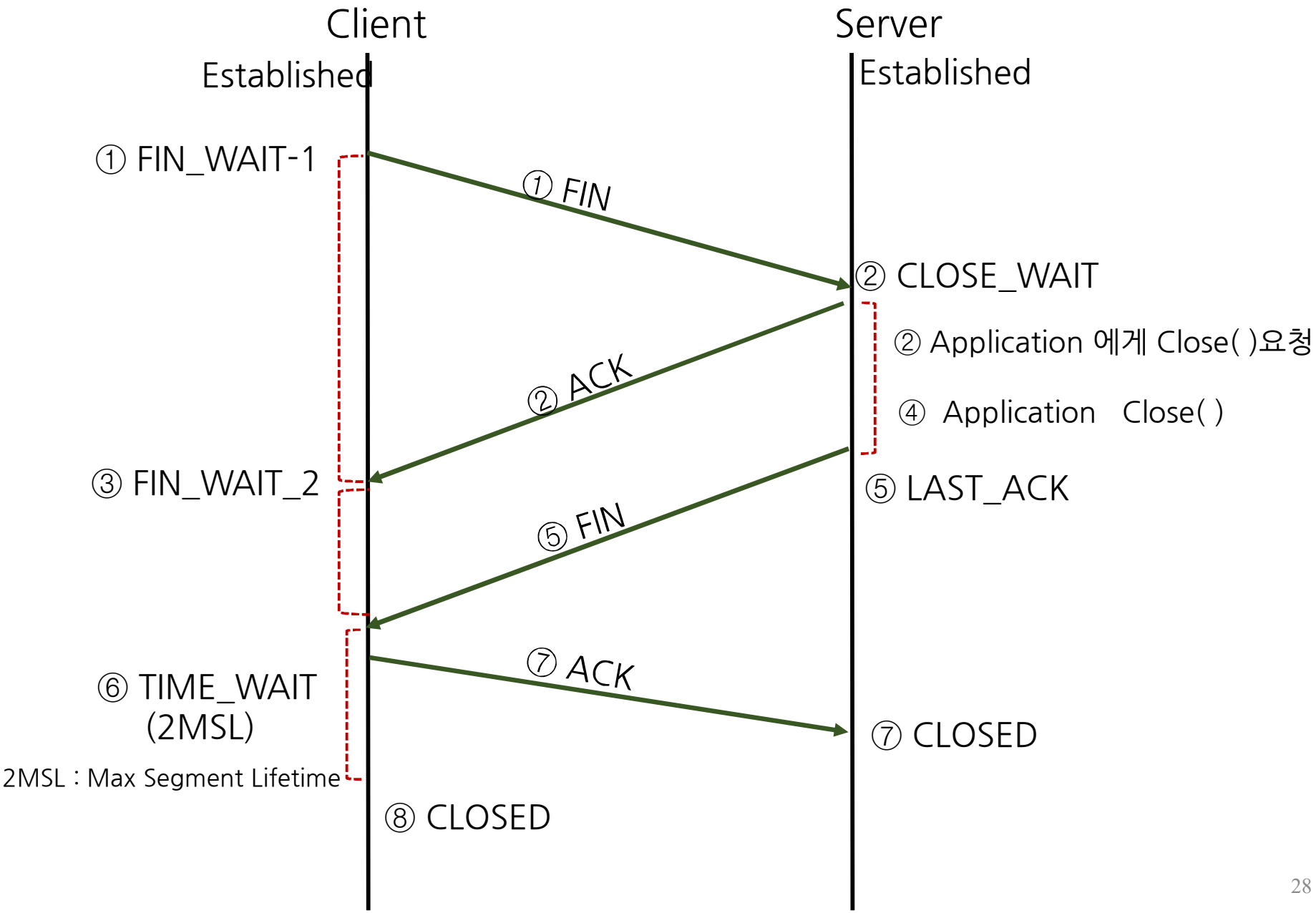
트래픽 흐름제어

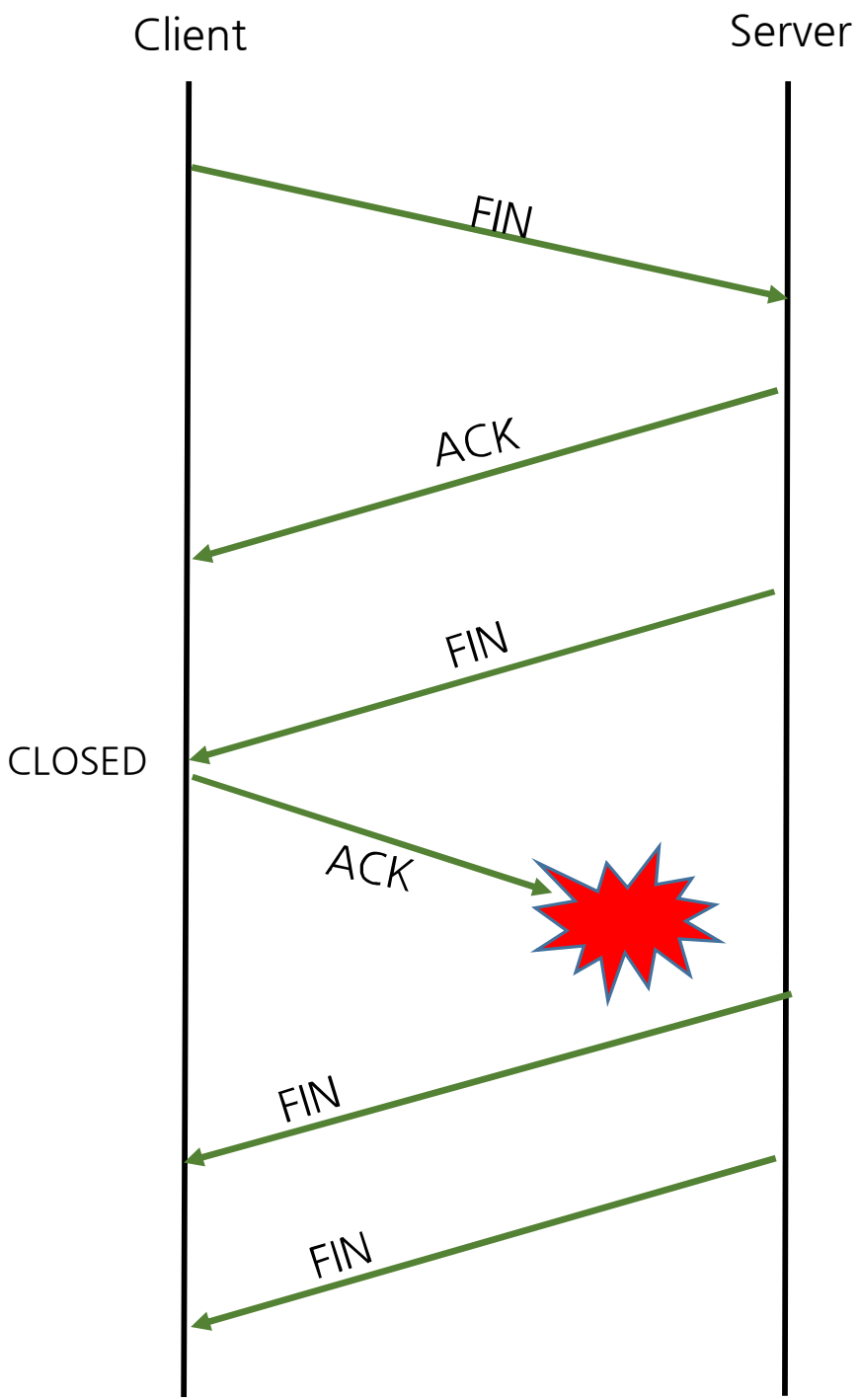

Window Size = 1500
SN=3000 (Data Size : 1400)
AN= 4400
Window Size = 5000
SN=4400 (Data Size : 1400)
SN=5800
SN=7200 (Data Size : 1400)
NAK= 5800
SN=5800 (Data Size : 1400)
AN= 7200
SN=8600
1000
SN=7200
1400
SN=5800
1400
SN=4400
1400
SN=3000
1400
SN=7200
1400
SN=????
SN=4400
1400
송신호스트
수신호스트

트래픽 흐름제어


1400 바이트
1000 바이트
Window size : 0 바이트
Keep Alive
Window size : 1000 바이트
800 바이트
200 바이트
송신호스트
수신호스트

TCP 연결 종료(4-way Handshake)

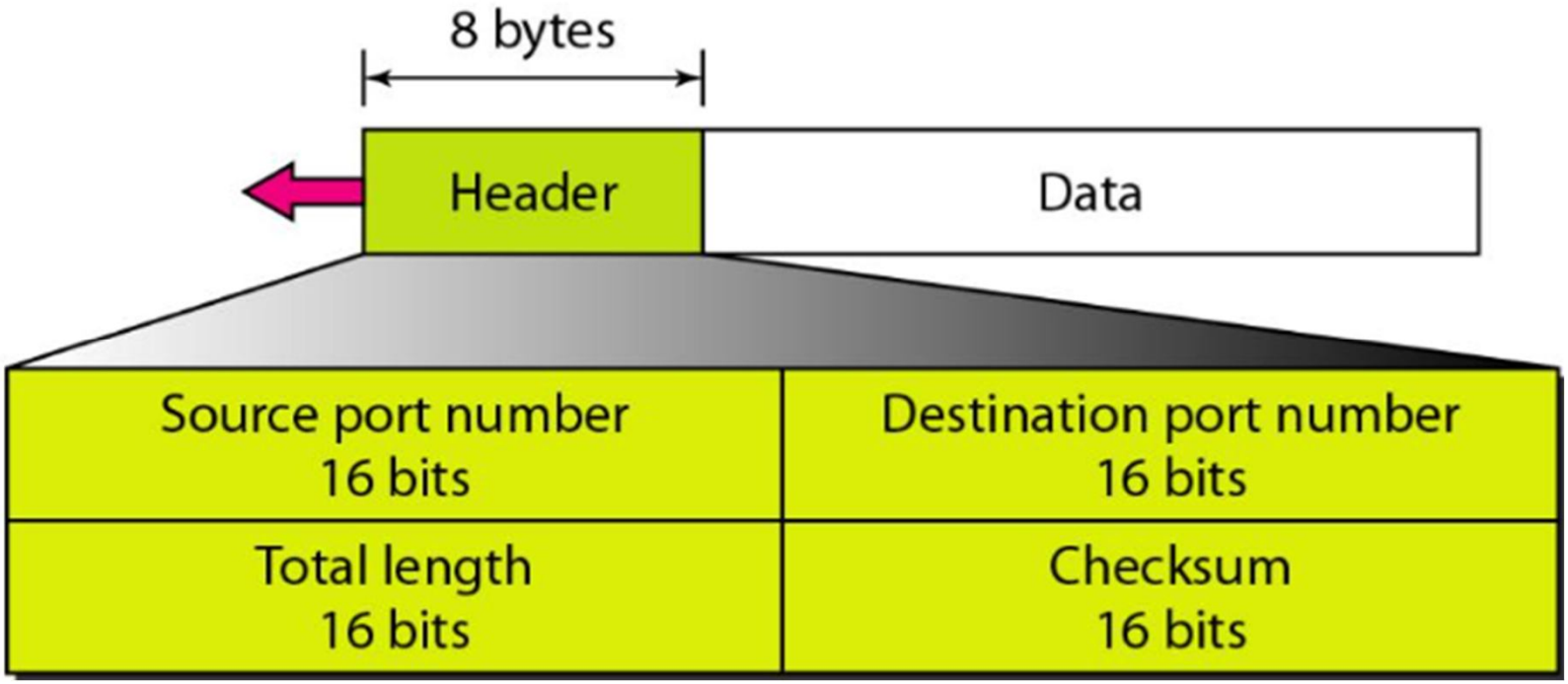




* Server에서는 소켓을 종료하지 못하게 됨

2) UDP(User Datagram Protocol) Header

- Data 전송을 위하여 사전에 필요한 Process가 없음
- Best-Effort Delivery
- 신뢰성 보장 못함



UDP

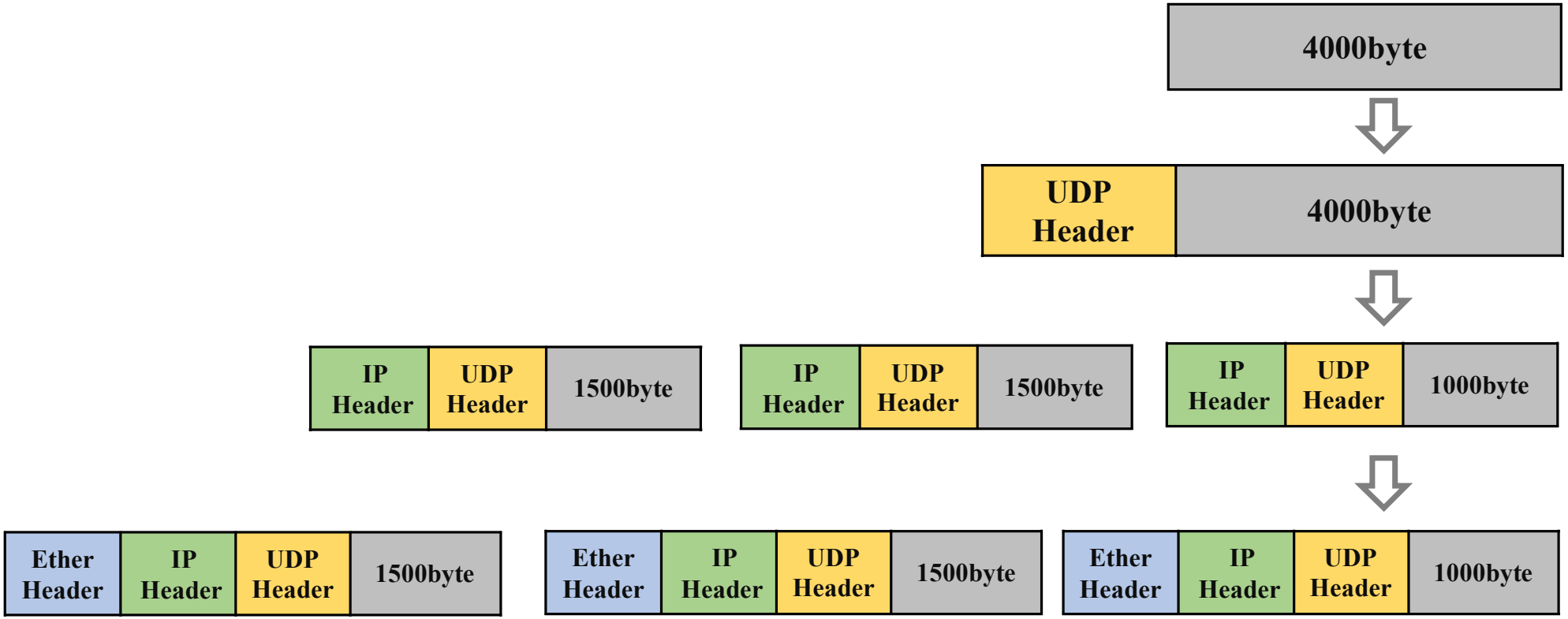
- Message-Oriented Transport Protocol
- 상위계층에서 payload 된 데이터를 단편화하지 않음
- 데이터 전송이 완료되지 전까지는 다른 데이터 흐름에 영향을 받지 않음

TCP

- Stream-Oriented Transport Protocol
- 상위계층에서 payload 된 데이터를 단편화
- 단편화를 통해 다수의 데이터들과 네트워크 자원을

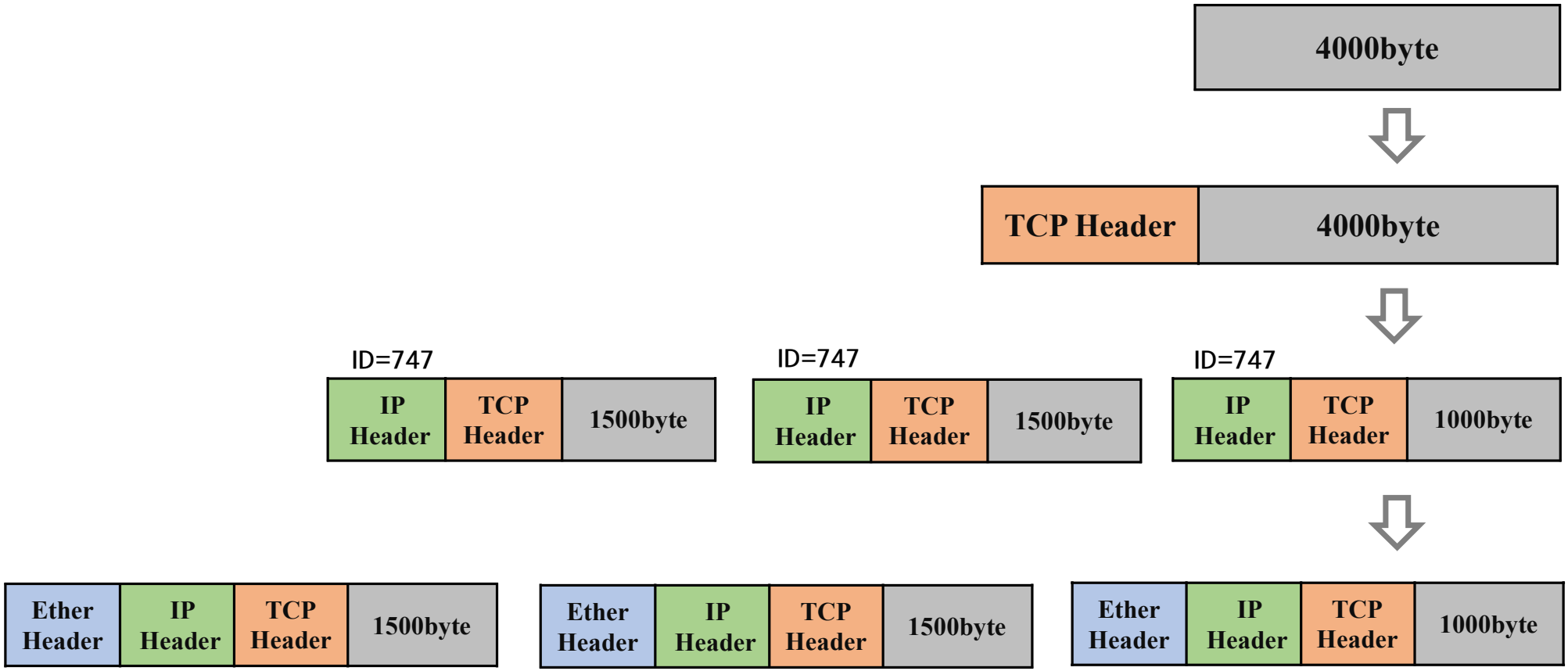
UDP 기반의 애플리케이션 데이터 전송

- Message-oriented transport protocol



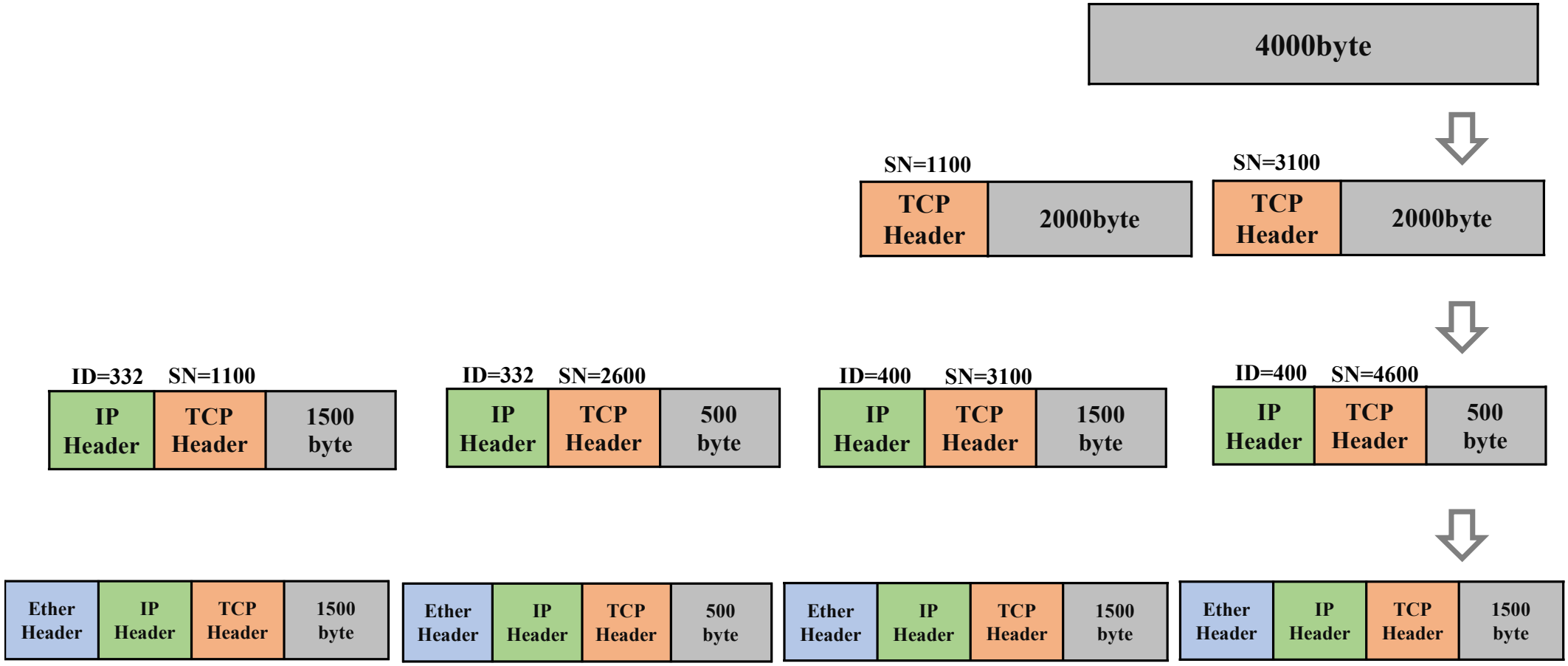
TCP 기반의 애플리케이션 데이터 전송

(예) MSS가 5000byte인 경우



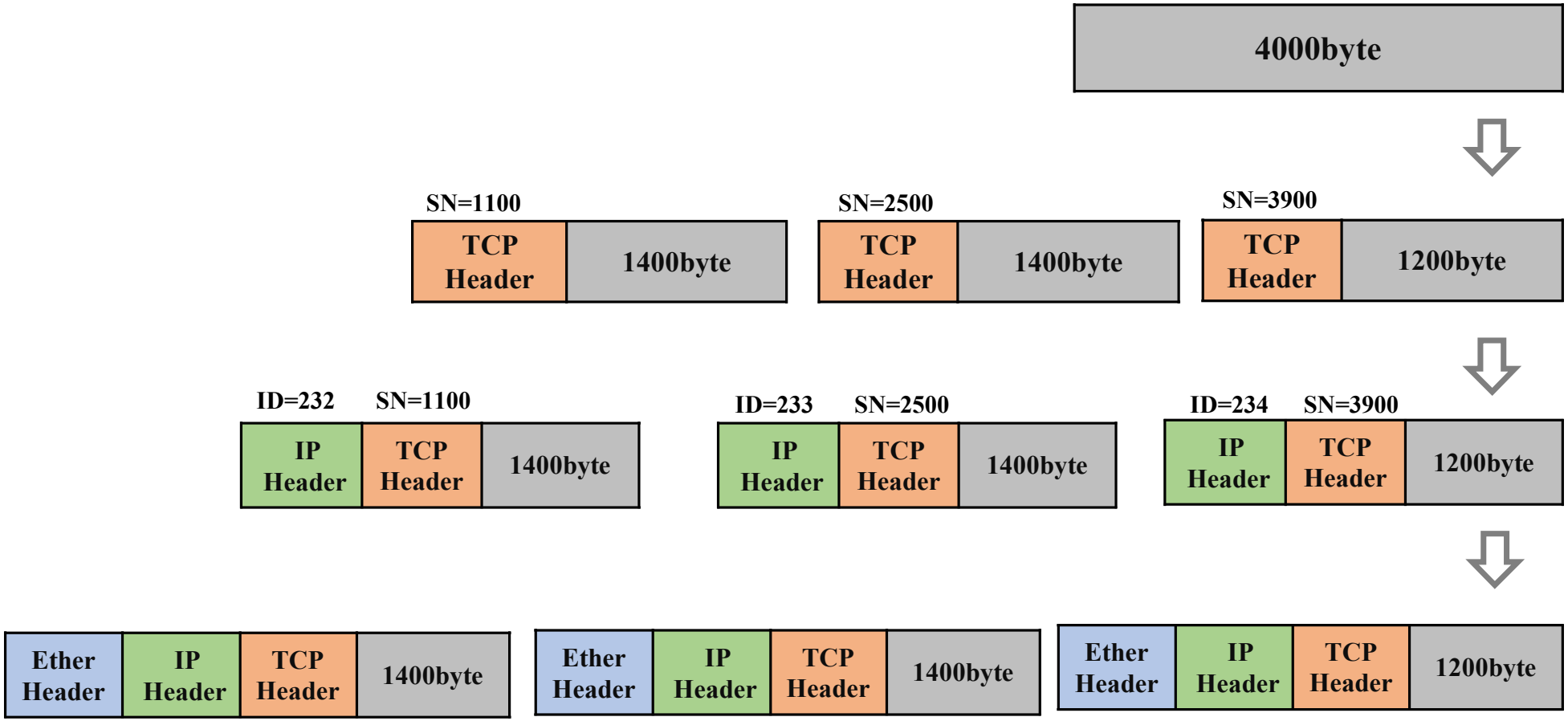
TCP 기반의 애플리케이션 데이터 전송

(예) MSS가 2000 byte인 경우



TCP 기반의 애플리케이션 데이터 전송

(예) MSS가 1400 byte인 경우

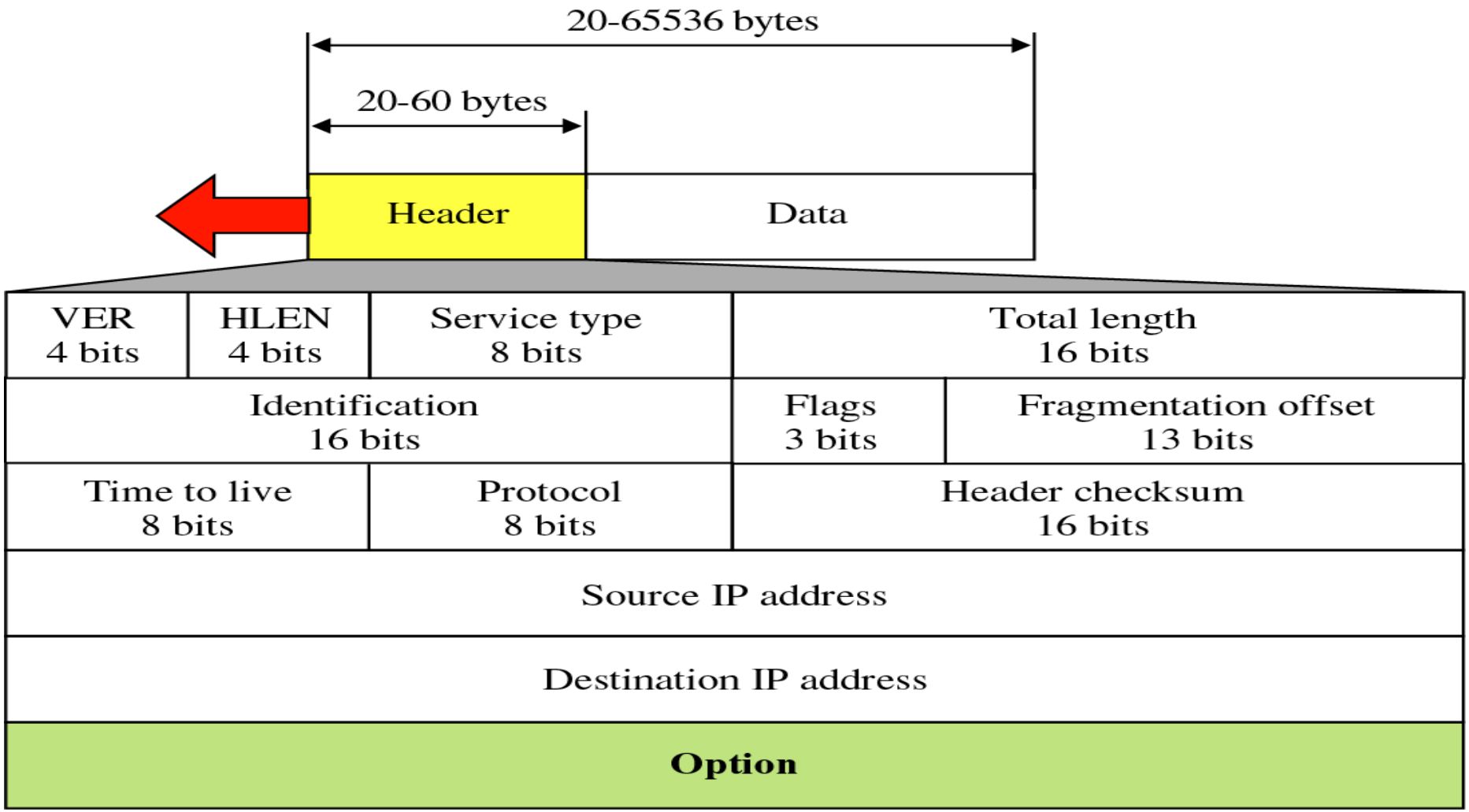


TCP와 UDP

TCP	UDP
Connection oriented	Connectionless
Sequencing 지원	Sequencing 지원하지 않음
Error control을 한다	Error control 하지 않음
Flow control을 한다	Flow control 하지 않음
Unicast 전송	Unicast, Multicast, Broadcast 전송
Full duplex	Half duplex
데이터 전송	실시간 Traffic 전송 (VoIP, Multimedia 등)

3) IP Header

- IP 헤더는 총 20Byte의 기본 길이와 옵션을 사용해 크기가 최대 60Byte까지 가질 수 있음



[IP Header Field]

- Version: IP프로토콜의 버전을 나타내는 4비트 정보
- IHL: Internet Header Length로 이 값에 5를 곱한 바이트 단위 크기가 IP 헤더의 크기
- ToS: Type of Service로 패킷의 처리 우선순위를 나타냄
- Total length: IP헤더와 Payload를 포함한 바이트 단위 길이
- Identification: 패킷 단편화 시 사용되는 식별자
- DF: Don’t Fragment. 단편화 금지 플래그
- MF: More Fragment. 이 패킷 이후 추가 단편이 있음을 알리는 플래그
- Fragment offset: 단편을 조립해 한 데이터로 만들 수 있도록 단편의 위치를 기술한 정보
- TTL: Time To Live 패킷이 한 Hop을 지날 때마다 감소되는 값. 0이 되면 패킷은 버려짐
- Protocol: IP헤더 다음 헤더가 무엇인지 알려줌
- Header checksum: 패킷에 대한 체크섬. 이 정보를 확인해 패킷의 손상 여부를 검출
- Source address: 패킷을 전송한 시스템의 IP주소
- Destination address: 패킷을 수신할 시스템의 IP주소

MTU (Maximum Transfer Unit)

- 네트워크 기기가 전송할 수 있는 최대 전송 단위
- 네트워크 환경에 따라 각각의 크기는 다음
- 현재 대부분의 네트워크 환경이기 때문에 MTU는 1500바이트로 통용되고 있음

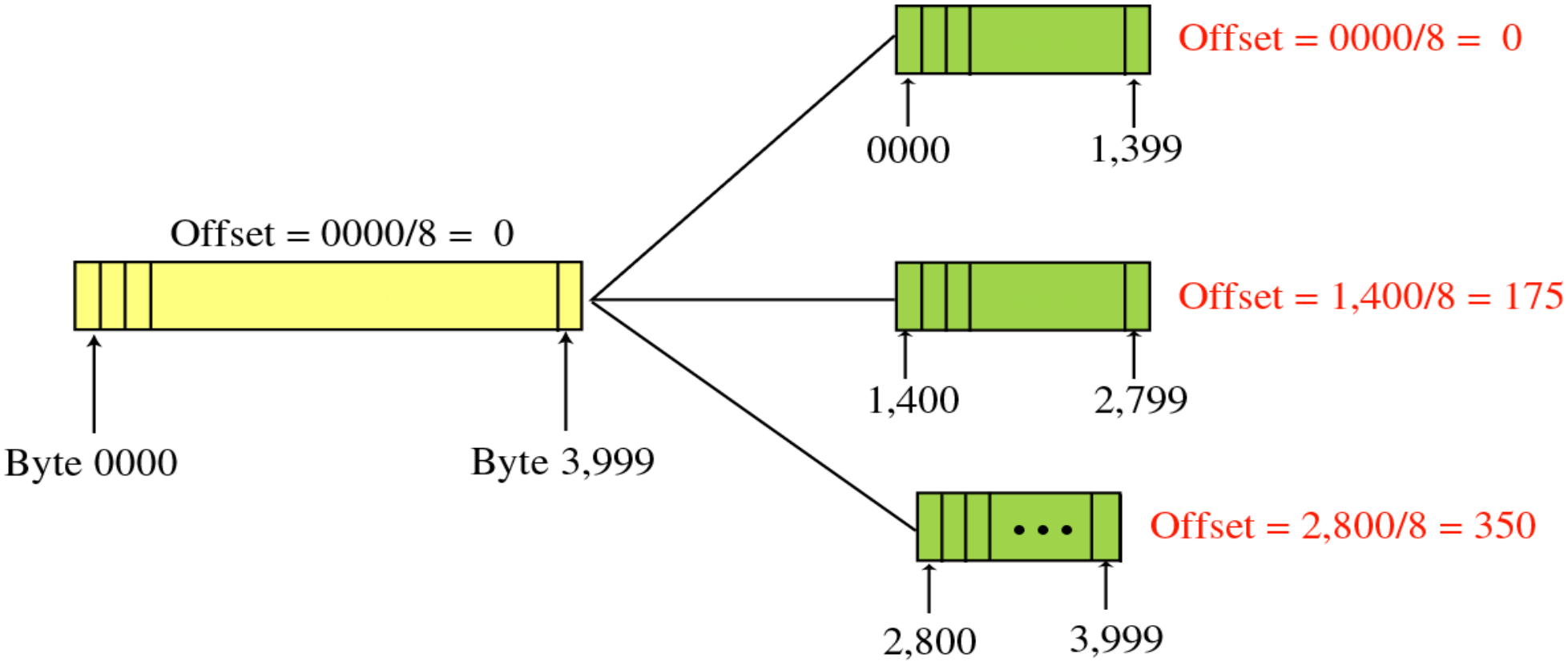
전송매체	MTU(bytes)
Internet IPv4 Path MTU	최소 68
Internet IPv6 Path MTU	최소 1280
Ethernet v2	1500
Ethernet LLC(Logical Link Control) SNAP (Subnetwork Access Protocol) PPPoE(P2P over Ethernet)	1492
Ethernet Jumbo Frames	1501~9216
WLAN(802.11)	7981
Token Ring(802.5)	4464
FDDI	4352

단편화(Fragmentation)

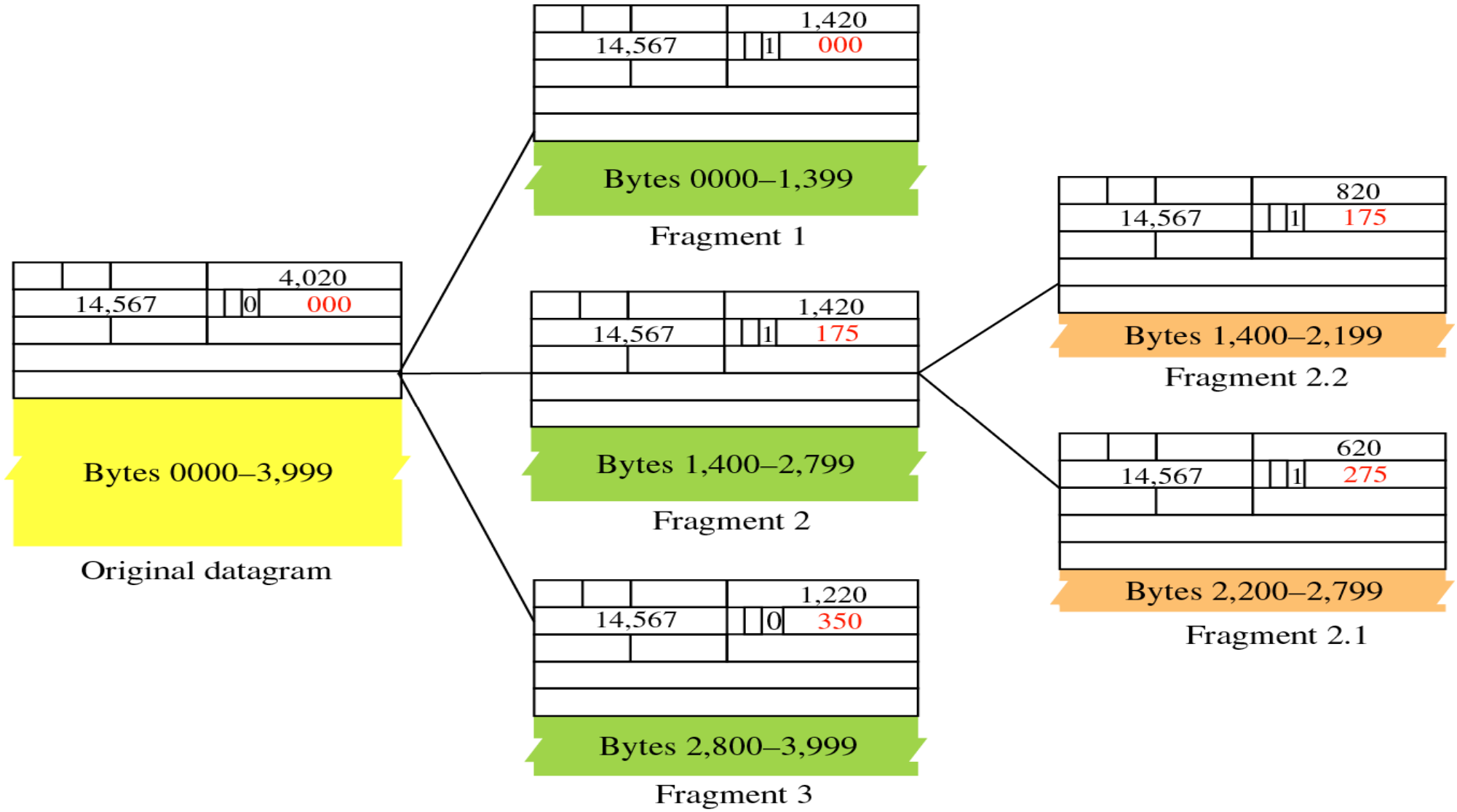
- MTU가 큰 네트워크에서 MTU가 작은 네트워크로 데이터그램이 전송될 경우 데이터그램은 나누어서 보내져야 함
- 데이터그램의 재조립은 최종목적지 호스트에 의해서만 수행
- 재조립으로 인해 발생하는 비효율성 때문에 전송 중 재조립 안됨
- 단편화와 관련된 필드 : Identification, Flag, Fragmentation offset

Fragmentation 예제

예를 들어 4000byte 데이터그램이 세 개로 단편화될 경우



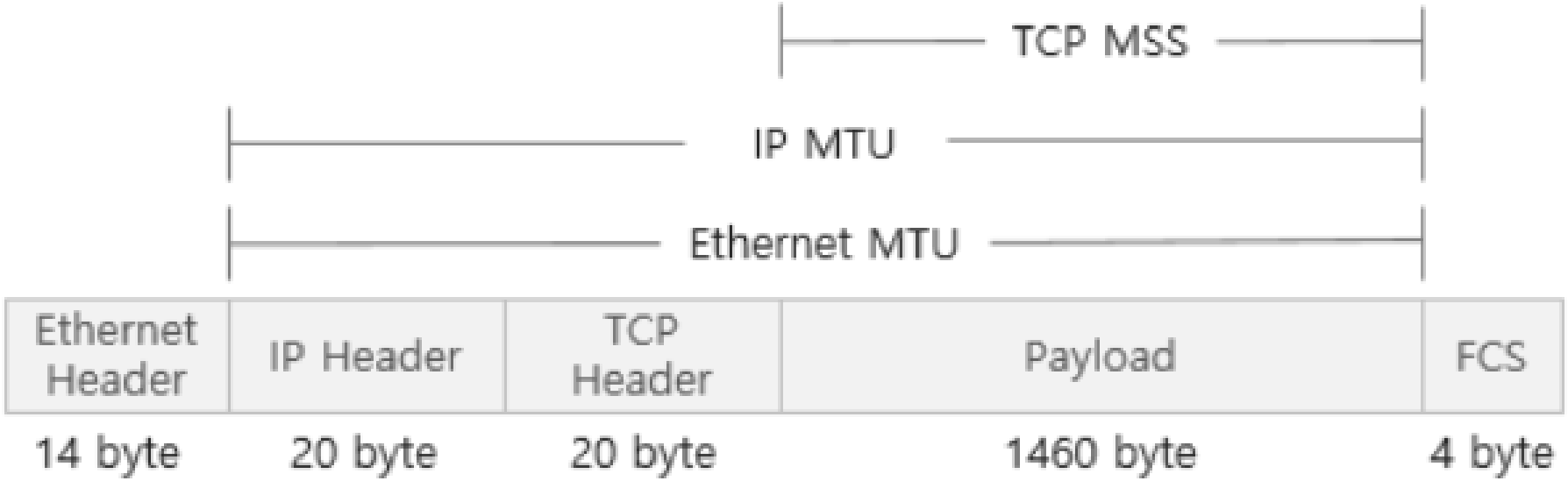
Fragmentation 예제



* MSS(Maximum Segment Size)

- TCP상에서 전송할 수 있는 사용자 데이터의 최대 크기

MSS = MTU - IP Header 크기 (최소 20 Byte) - TCP Header 크기 (최소 20 Byte)



[참고] 명령어 netsh

- 마이크로소프트의 유틸리티
- 로컬 또는 원격 구성의 네트워크 설정을 변경

```
C:₩>netsh ?

사용법: netsh [-a 별칭 파일] [-c 컨텍스트] [-r 원격 컴퓨터]
              [-u [DomainName₩]UserName] [-p 암호 | *] [명령 | -f 스크립트 파일]

다음 명령을 사용할 수 있습니다.

이 컨텍스트에 있는 명령:
?               - 명령 목록을 표시합니다.
add             - 항목 목록에 구성 항목을 추가합니다.
advfirewall     - `netsh advfirewall' 컨텍스트의 변경 내용입니다.
branchcache     - `netsh branchcache' 컨텍스트의 변경 내용입니다.
bridge          - `netsh bridge' 컨텍스트의 변경 내용입니다.
delete          - 항목 목록에서 구성 항목을 삭제합니다.
dhcpclient      - `netsh dhcpclient' 컨텍스트의 변경 내용입니다.
dnsclient       - `netsh dnsclient' 컨텍스트의 변경 내용입니다.
dump            - 구성 스크립트를 표시합니다.
```

관리자: 명령 프롬프트

```
C:₩>
C:₩>net int ipv4 set global ?
이 명령에 대한 구문:

NET
    [ ACCOUNTS | COMPUTER | CONFIG | CONTINUE | FILE | GROUP | HELP |
      HELPMSG | LOCALGROUP | PAUSE | SESSION | SHARE | START |
      STATISTICS | STOP | TIME | USE | USER | VIEW ]

C:₩>
```

C:₩>netsh interface show interface

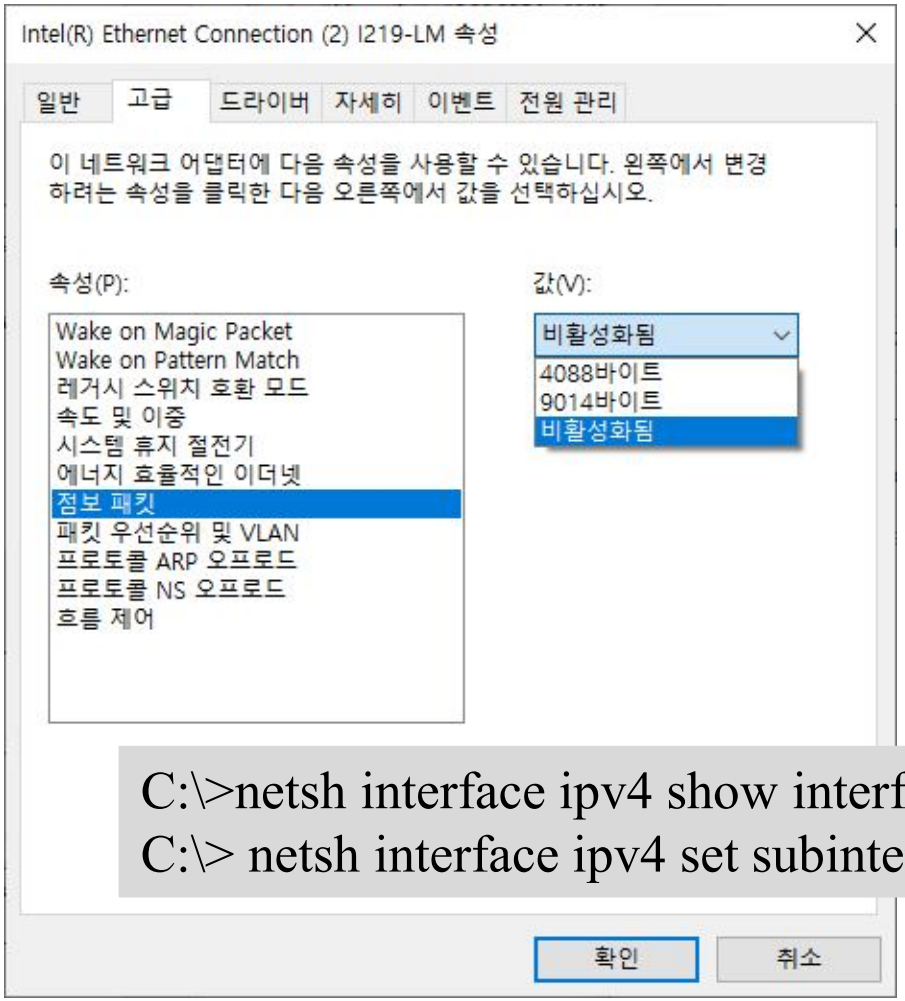
C:₩>netsh interface ipv4 show global

C:₩>netsh interface ipv4 show interfaces

* MTU 설정 변경

C:₩> netsh interface ipv4 set subinterface “6" mtu=9000 store=persistent

C:₩>netsh interface ipv4 show global



C:\>netsh interface ipv4 show interfaces
C:\> netsh interface ipv4 set subinterface “6" mtu=9000 store=persistent

```
C:₩>netsh interface ipv4 show interfaces

색인     메트릭          MTU          상태             이름
---  ----------  ----------  ------------  ---------------------------
  1          75  4294967295  connected     Loopback Pseudo-Interface 1
  6          35        1500  connected     이더넷 2
  5           5        1500  disconnected  이더넷 3
  9           5        1500  disconnected  이더넷 4
  8          35        1500  connected     VMware Network Adapter VMnet8
```

TTL(Time To Live)

- 패킷 수명을 제한하기 위해 데이터그램이 통과하는 최대 홉(hop)수를 지정
- 패킷 전달과정에서 라우터와 같은 전송장비를 통과 할 때마다 TTL값 감소
- TTL이 0이 되면 라우터에서 폐기하여 불필요한 패킷이 네트워크에 방치 되는 것을 방지
- OS종류와 버전에 따라 TTL 값이 다름

OS/Version	TCP TTL	UDP TTL
Linux	64	64
HP/UX 10.01	64	64
Solaris 2.z	255	255
Window Server 2008	128	128
Windows 10	64	64

* TTL 값 변경

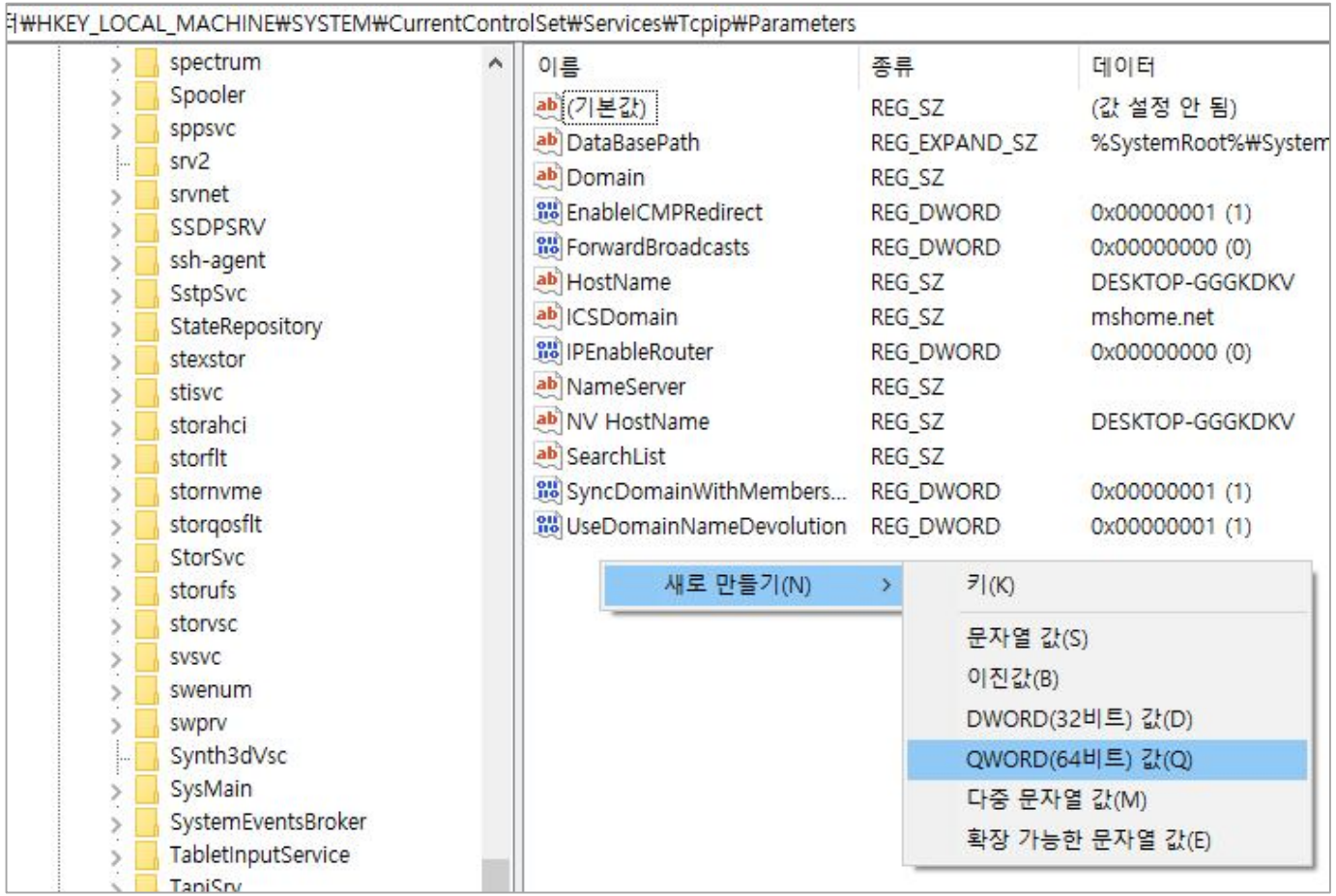
HKEY_LOCAL_MACHINE

₩SYSTEM₩CurrentControlSet₩Services₩Tcpip₩Parameters

Name: DefaultTTL

Type: REG_DWORD

Valid Range: 1-255



C:₩>netsh interface show interface

C:₩>netsh interface ipv4 show global

C:₩>netsh interface ipv4 show interfaces

* TTL 설정 변경

C:₩> netsh interface ipv4 set global defaultcurhoplimit=64

C:₩>netsh interface ipv4 show global

Header checksum

- 헤더의 오류를 검증하기 위해 사용
- 계상방식은 version 필드 값부터 마지막 필드인 목적지 IP 필드 값까지 모두 더함

  - Version 필드~ 목적지 IP 필드(Checksum 필드 제외)

```
Internet Protocol Version 4, Src: 10.40.219.42, Dst: 10.40.201.225
    0100 .... = Version: 4
    .... 0101 = Header Length: 20 bytes (5)
  > Differentiated Services Field: 0x00 (DSCP: CS0, ECN: Not-ECT)
    Total Length: 64
    Identification: 0x1042 (4162)
  > Flags: 0x40, Don't fragment
    ...0 0000 0000 0000 = Fragment Offset: 0
    Time to Live: 254
    Protocol: TCP (6)
    Header Checksum: 0xb319 [correct]
    [Header checksum status: Good]
    [Calculated Checksum: 0xb319]
    Source Address: 10.40.219.42
    Destination Address: 10.40.201.225

0000  00 26 b9 48 ff 97 00 10  db ff 10 01 08 00 45 00
0010  00 40 10 42 40 00 fe 06  b3 19 0a 28 db 2a 0a 28
0020  c9 e1 c6 b0 00 50 d6 87  40 de 00 00 00 00 b0 02
0030  ff ff 92 d4 00 00 02 04  05 b4 01 03 03 05 01 01
0040  08 0a 13 ad f8 b8 00 00  00 00 04 02 00 00
```

Version/ToS : 4500
Total Length : 0040
ID : 1042
Flags : 4000
TTL/Protocol : fe06
SourceIP : 0a28 + db2a
DestinationIP : 0a28+c9e1

34ce3

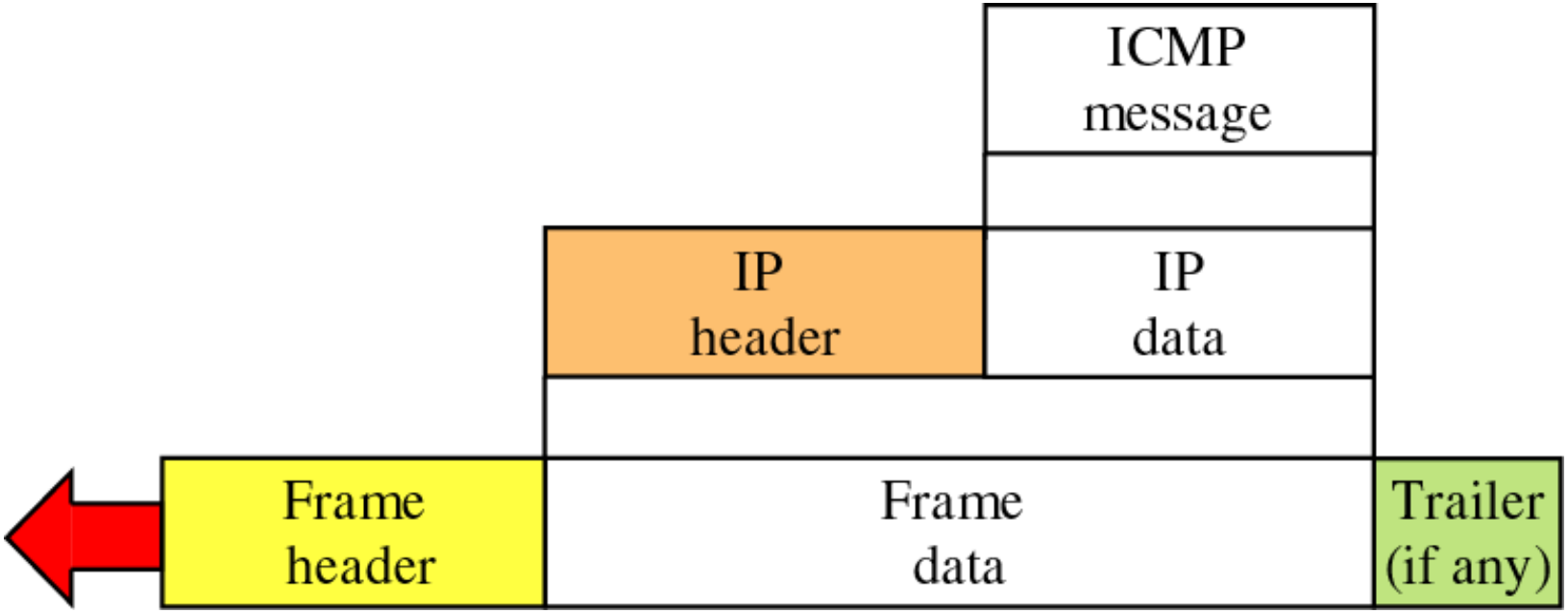
3+ 4ce3 = 4ce6

4ce6 = 0100 1100 1110 0110 (2진화)
= 1011 0011 0001 1001 (1의보수)
= B319

4) ICMP(Internet Control Message Protocol)

- IP Protocol은 송신시스템과 수신시스템 사이의 패킷을 최적의 경로를 통해 전달하는 것이 주된 목적
- IP Protocol은 신뢰성이 없고 비연결형 Protocol
- IP Protocol은 에러 발생 원인이나 진단 기능 및 상황 정보를 지원하지 않음

  ➔ ICMP Protocol Support (IP의 단점 보완)



❶ ICMP 메시지 종류

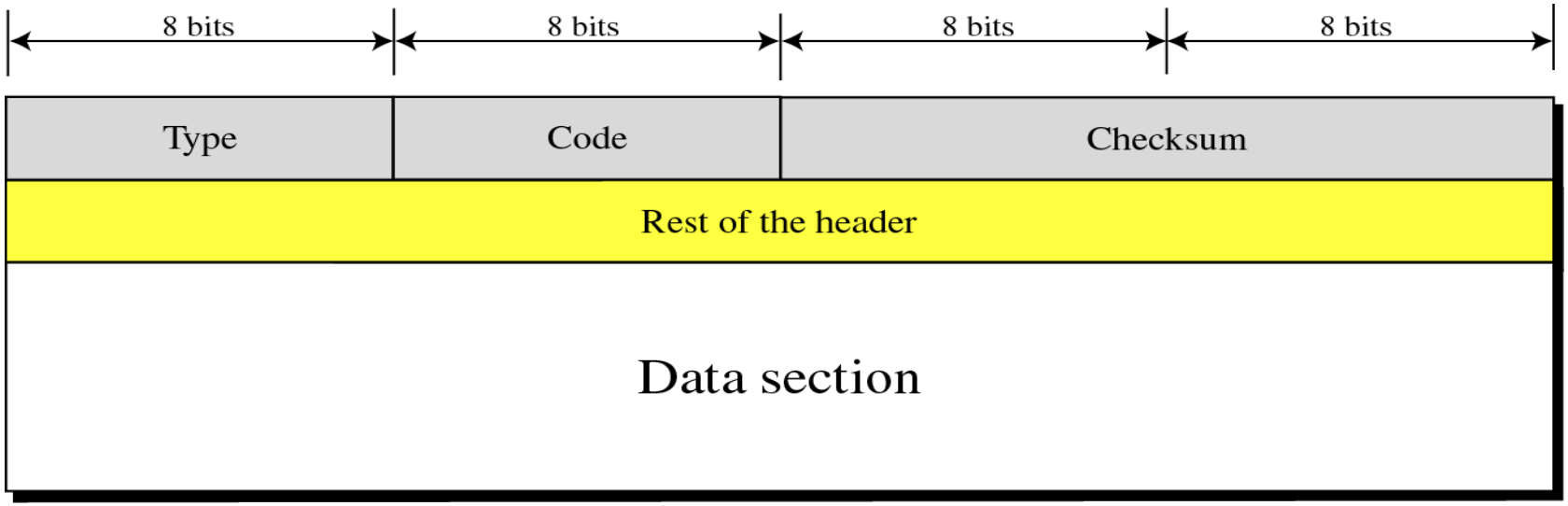
- 오류 보고 메시지 (Error Reporting Massage)

  - IP 패킷 처리 도중 발생한 문제 보고

- 질의 메시지 (Query Massage)

  - 다른 호스트로부터 특정 정보 획득
  - 네트워크 문제 진단

❷ ICMP Header



- 총 8Byte로 구성
- Type : ICMP의 업무, 즉 어떠한 용도로 사용되는 ICMP를 나타냄
- Code : ICMP Type의 세부 내용을 나타내며 이 부분은 Type과 조합을 이루어 ICMP 메시지의 목적과 용도를 나타냄
- Checksum : ICMP 메시지의 이상 유무 판단

❸ ICMP Code

Type	Code	Description	Query	Error
0	0	Echo Reply (ping reply)	*	
3		Destination unreachable		
	0	Network unreachable - 지정된 Network이 없을 경우 라우터가 생성		*
	1	Host unreachable - 마지막 라우터가 호스트와 통신 불가		*
	2	Protocol unreachable - 목적지 호스트가 생성하며 IP 헤더의 프로토콜 필드로 식별된 상위 프로토콜을 사용할 수 없을 경우		*
	3	Port unreachable - 목적지 호스트가 생성		*
	4	Fragmentation needed for DE = 1 - MTU크기보다 데이터그램이 커서 전달할 수 없을 경우 라우터에서 생성		*
	5	Source Route Failed		*

❸ ICMP Code

Type	Code	Description	Query	Error
4	0	Source quench		*
5		Redirect		
	0	Redirect for network		*
	1	Redirect for host		*
	2	Redirect for type of service and network		*
	3	Redirect for type of service and host		*
8	0	Echo request (ping 요청)	*	
9	0	Router advertisement	*	
10	0	Router solicitation	*	

* 에러 보고 메시지

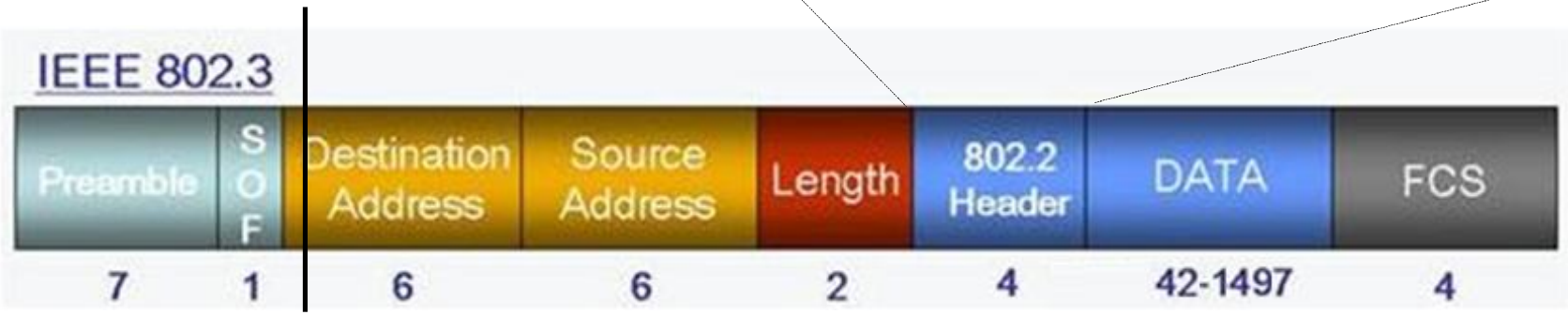
타입	메시지 이름	설명
3	Destination Unreachable (목적지 도착 불가 메시지)	• 라우터가 패킷을 라우팅 할 수 없거나 호스트가 패킷을 전달할 수 없을 때 해당 패킷은 전달이 불가하다고 판단한 라우터나 호스트가 폐기하고 출발지 호스트에 '목적지도착불가'메시지를 전달
4	Source Quench (발신지 억제)	• 송신자에게 링크 혼잡으로써 패킷이 폐기되었음을 알려줌 혼잡상황을 알려줄뿐 혼잡원인을 제공하지 않음 • 발신지 억제 메시지를 수신한 시스템 자체에 혼잡을 유발했다고 단정할 수는 없음
5	Redirection (재지정)	• 특정 목적지에 대해 더 나은 경로를 갖고 있는 다른 라우터가 있음을 알려줌
11	Time exceeded (시간초과)	• TTL 만료로 패킷이 폐기 되었음을 알림

5) Ethernet Frame 구조

❶ Ethernet Frame (DIX 2.0)

❷ IEEE 802.3 & IEEE 802.2

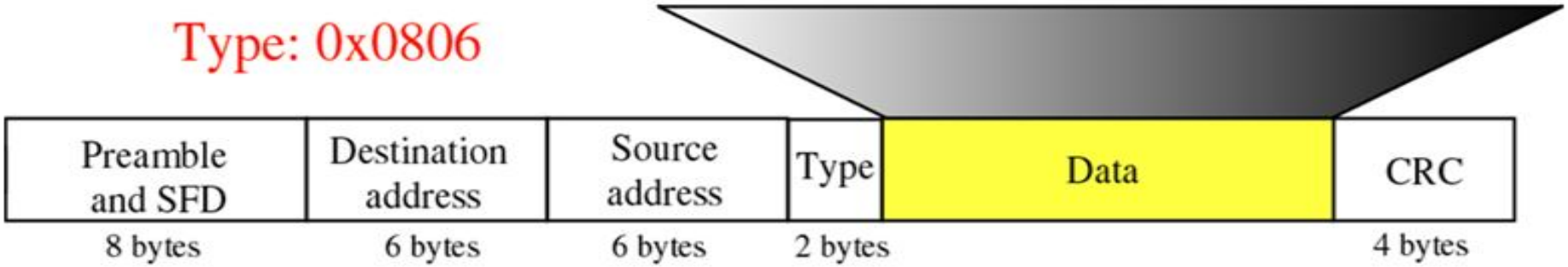
DSAP address	SSAP address	Control	Information
8 bits	8 bits	8 or 16 bits	M*8 bits



6) ARP(Address Resolution Protocol)

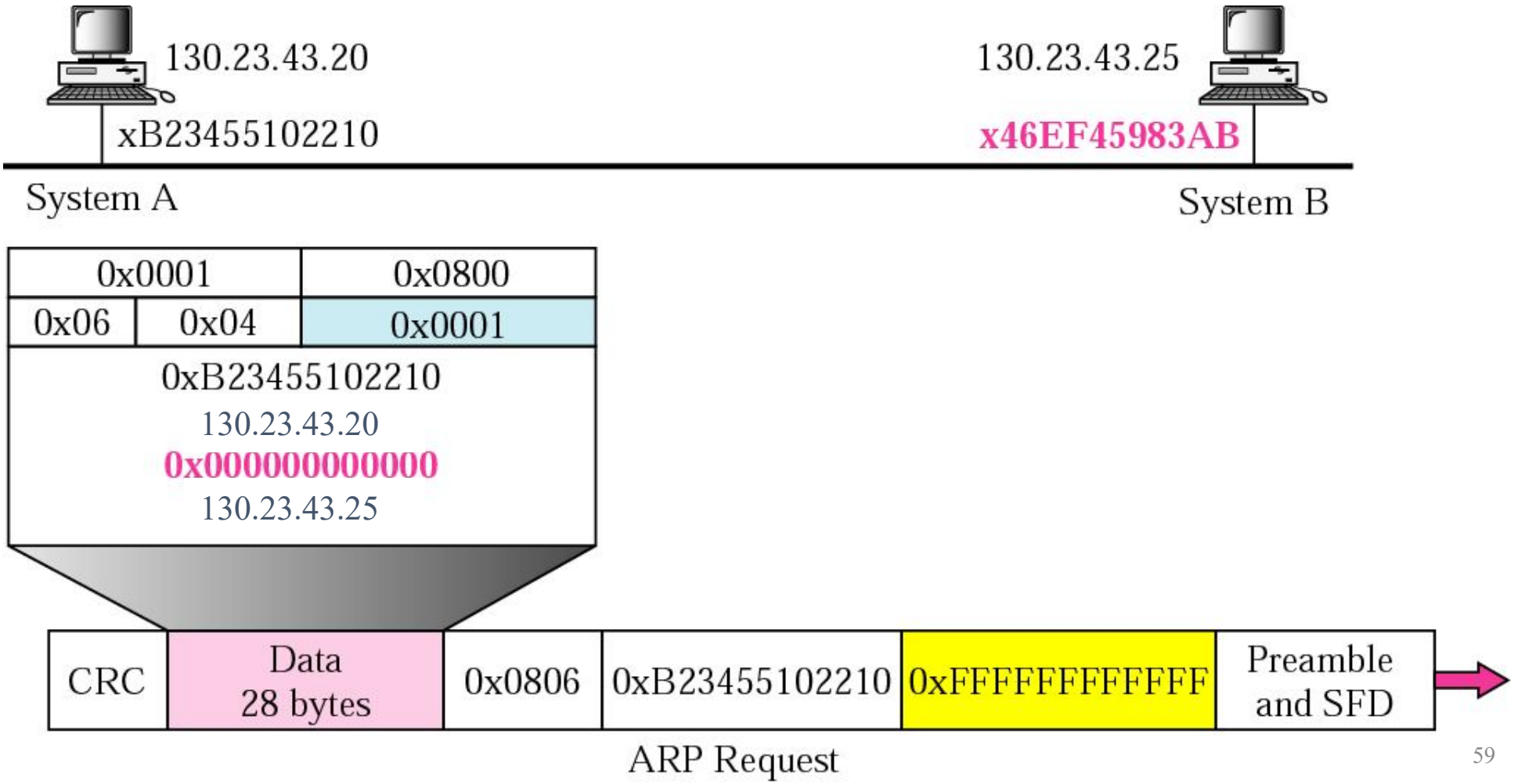
Hardware Type		Protocol Type
Hardware length	Protocol length	Operation Request 1, Reply 2
Sender hardware address (For example, 6 bytes for Ethernet)		
Sender protocol address (For example, 4 bytes for IP)		
Target hardware address (For example, 6 bytes for Ethernet) (It is not filled in a request)		
Target protocol address (For example, 4 bytes for IP)		

Type: 0x0806



ARP Request Packet

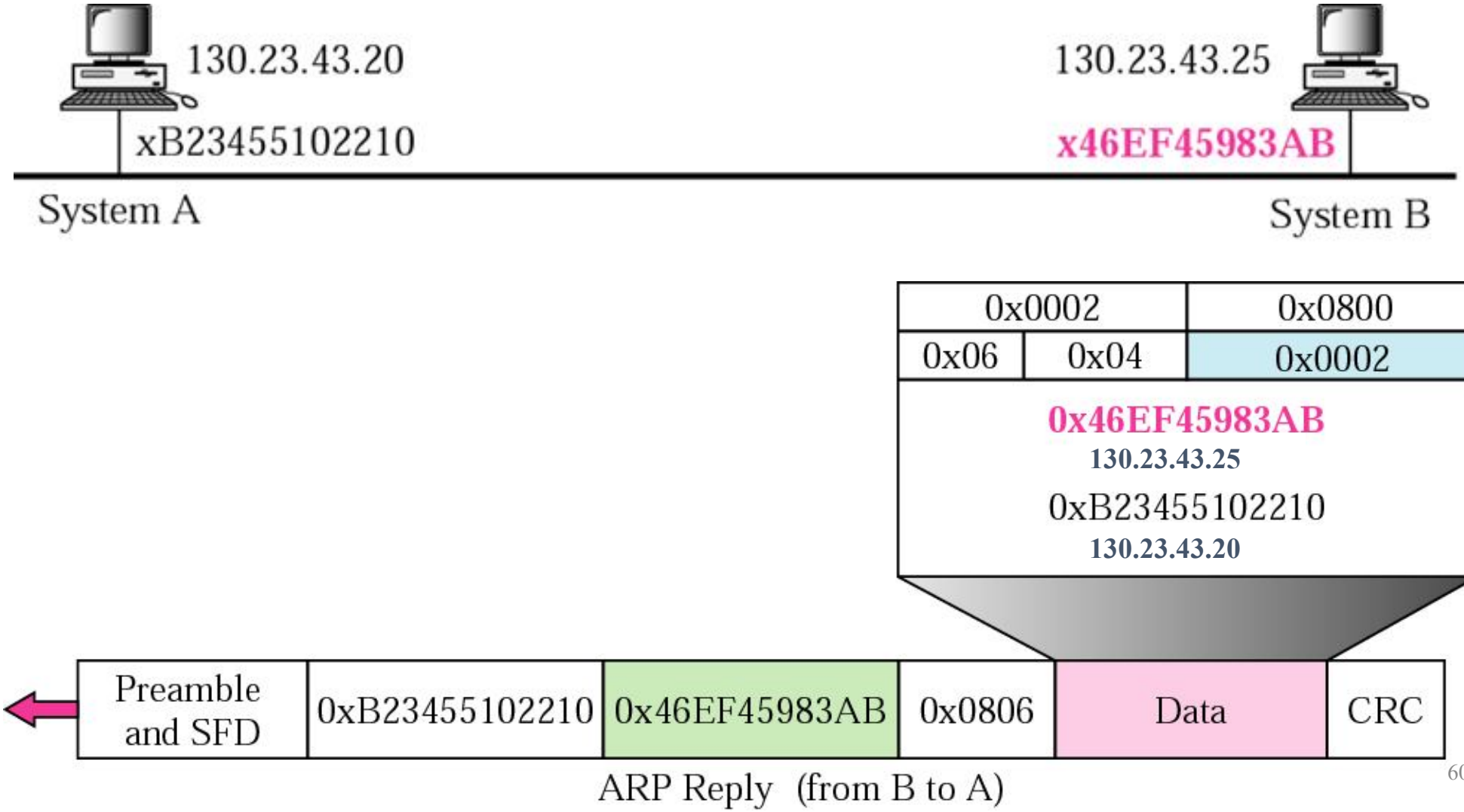
ARP Cache Table



ARP Reply Packet

ARP Cache Table

130.23.43.20 B234-5510-2210



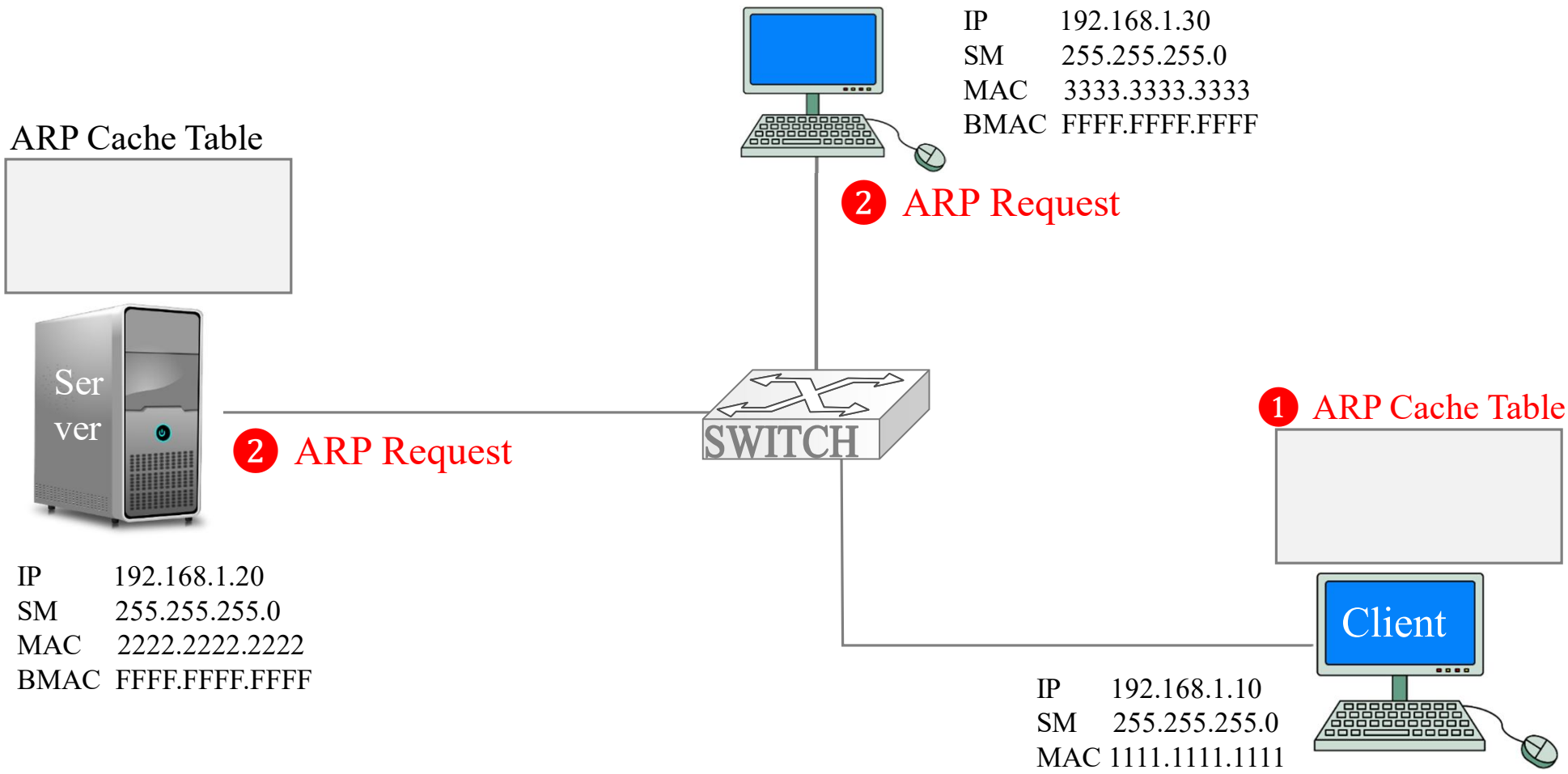
ARP 수행 후 캐쉬 테이블

ARP Cache Table

130.23.43.25 46EF-4598-3AB

ARP Cache Table

130.23.43.20 B234-5510-2210



ARP Request

수MAC	송MAC	Protocol	ARP Header	
FFFF.FFFF.FFFF	1111. 222.3333	0x0806	1111.1111.1111 /192.168.1.10 **0000.0000.0000/192.168.1.20**	송신지 수신지

수MAC	송MAC	송IP	수IP	송Port	수Port	전송데이타
????	1111. 1111.1111	192.168.1.10	**192.168.1.20**	50030	80	Get /www.test.com

	ARP Header	Protocol	송MAC	수MAC
송신지 수신지	2222.2222.2222 /192.168.1.20 1111.1111.1111 /192.168.1.10	0x0806	2222.2222.2222	1111.1111.1111

ARP Reply

ARP Cache Table

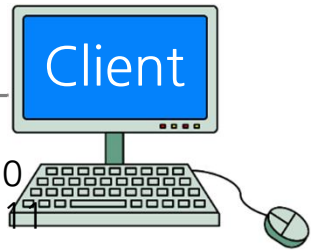
192.168.1.10 1111.1111.1111

IP 192.168.1.20
SM 255.255.255.0
MAC 2222.2222.2222
BMAC FFFF.FFFF.FFFF

ARP Cache Table

192.168.1.20 2222.2222.2222

ARP Reply

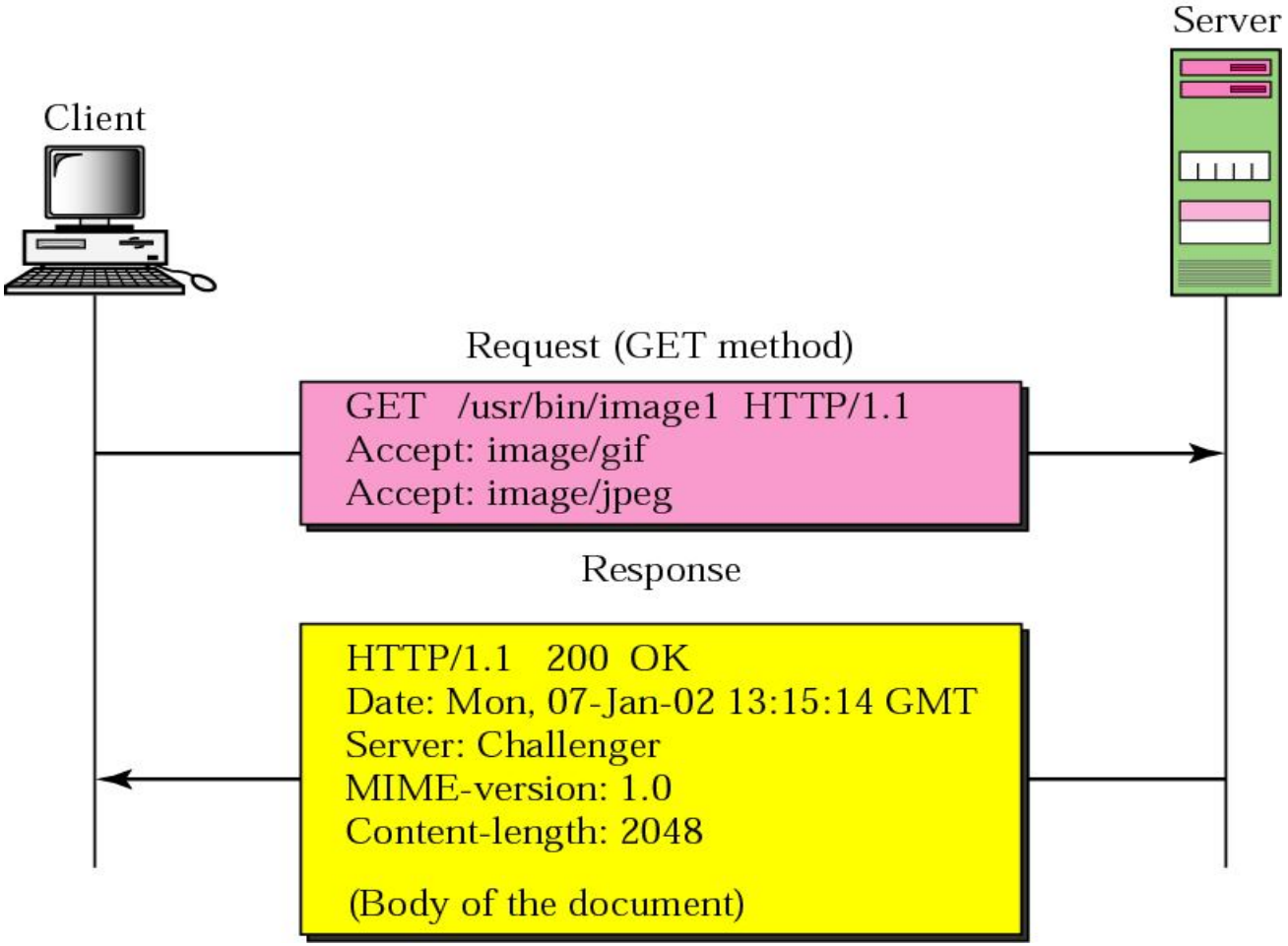


IP 192.168.1.10
SM 255.255.255.0
MAC1111.1111.1111

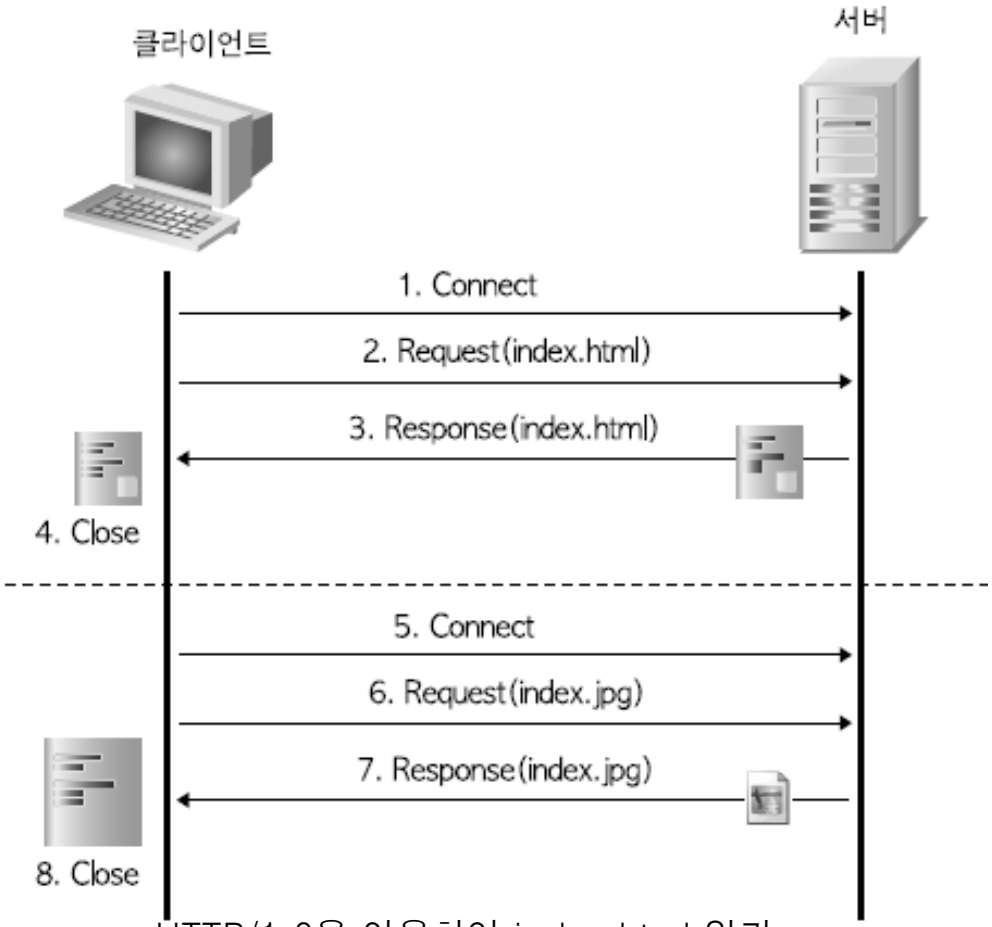
수MAC	송MAC	송IP	수IP	송Port	수Port	전송데이타
222.2222.2222	1111. 1111.1111	192.168.1.10	192.168.1.20	50030	80	Get /www.test.com

7) HTTP

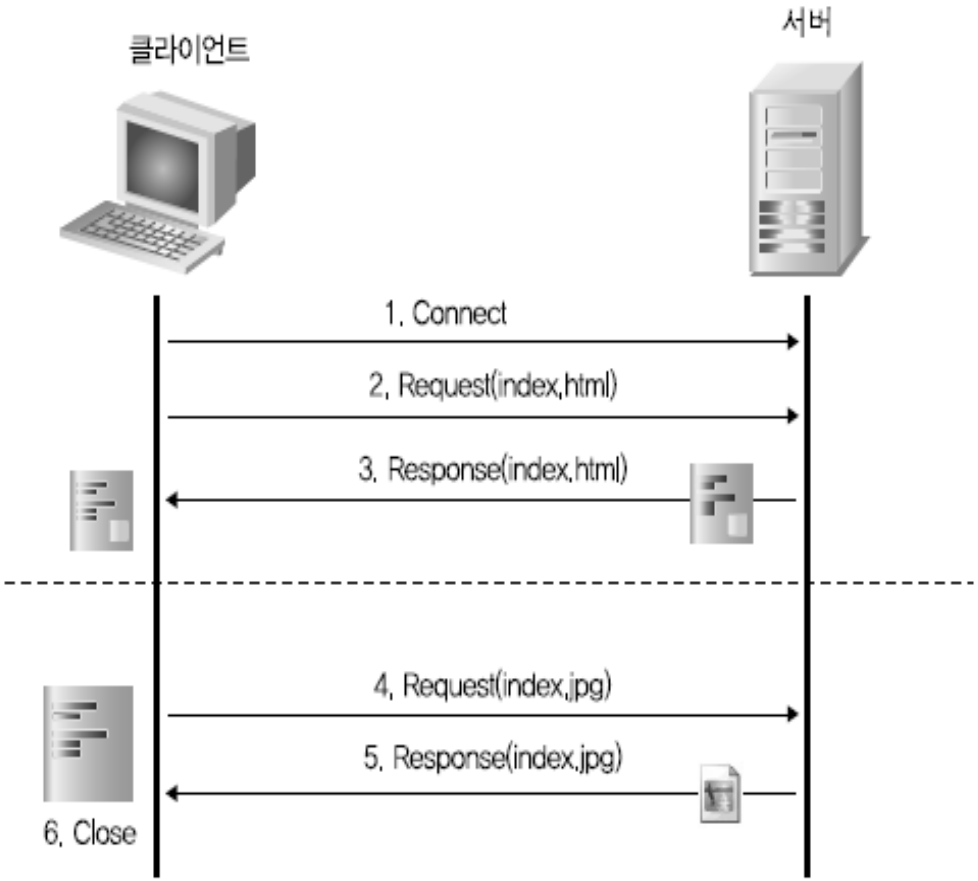
- Message 종류
  - Request : 요청 Message (Client ➔ Server)
  - Response : 응답 Message (Server ➔ Client)



- HTTP 버전

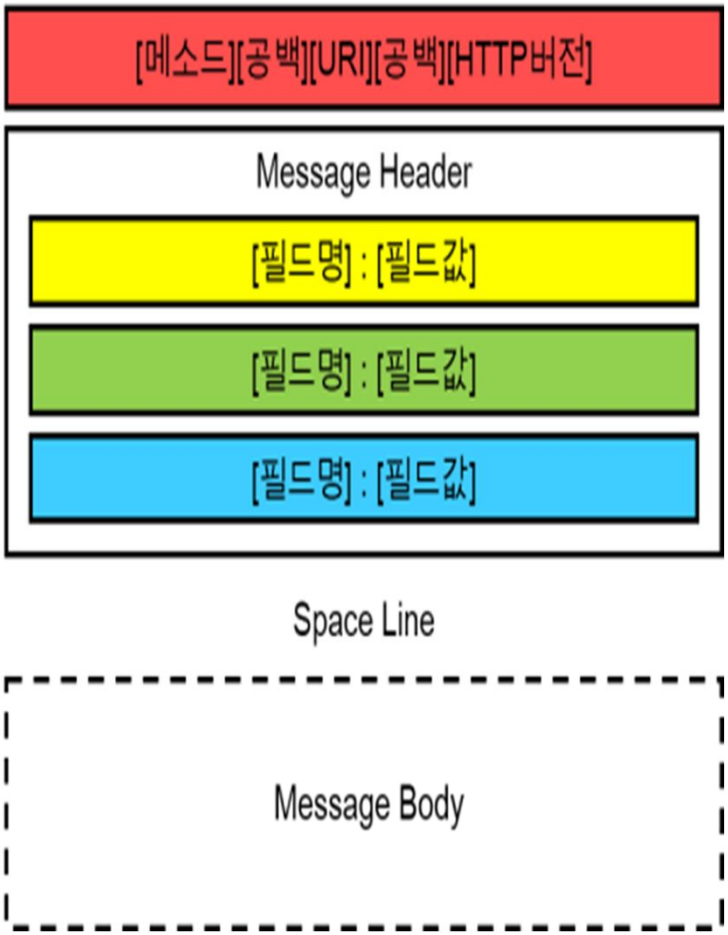


HTTP/1.0을 이용하여 index.html 읽기



HTTP/1.1을 이용하여 index.html 읽기

HTTP Request Message Format




```
GET / HTTP/1.1
Accept: text/html, application/xhtml+xml, */*
Accept-Encoding: gzip, deflate
Cookie: HSID=AaxlkKoV2snlEi6UQ; SSID=AAYVu_evC0Tiu3aVc;
APISID=JEWp0eojRTLFtYKJ/ACgEWh_0mL8Li_-fl;
SAPISID=kabRBO-uT0ebDcfc/A2byDX--FwN649tHw
Host: www.google.com
Connection: Keep-Alive
Accept-Language: ko-KR
User-Agent: Mozilla/5.0 (compatible; MSIE 9.0; Windows NT 6.1; Trident/5.0)
```

<<HTTP Request Method>>

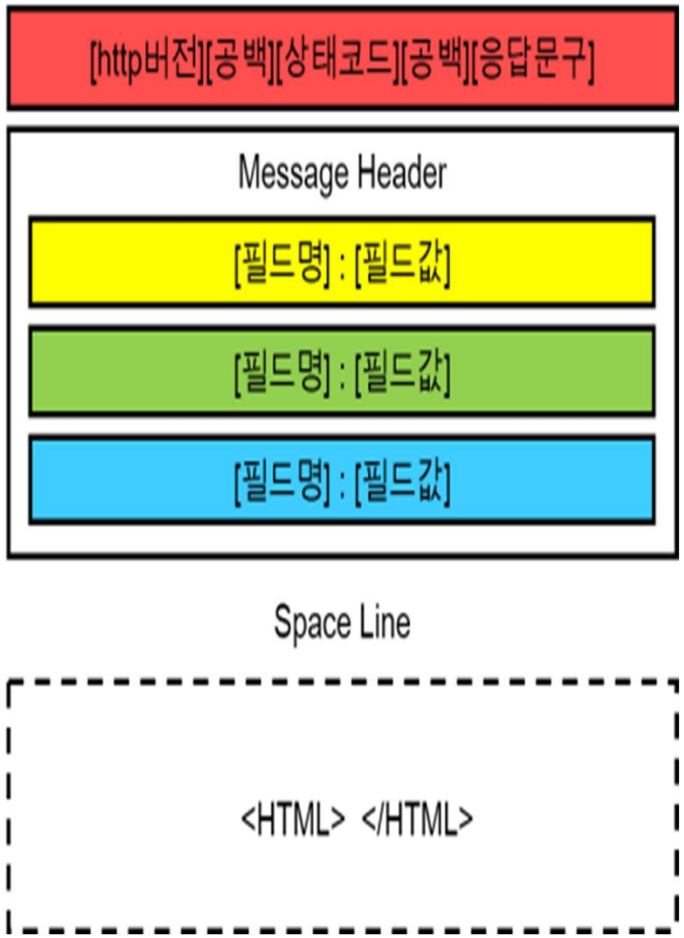
- GET
- POST
- HEAD
- OPTIONS
- PUT
- DELETE
- TRACE

POST /form/entry HTTP/1.1

❶ 메소드 ❷ 요청URI ❸ 프로토콜 버전

```
GET / HTTP/1.1
Accept: text/html, application/xhtml+xml, */*
Accept-Encoding: gzip, deflate
Cookie: HSID=AaxlkKoV2snlEi6UQ; SSID=AAYVu_evC0Tiu3aVc;
APISID=JEWp0eojRTLFtYKJ/ACgEWh_0mL8Li_-fl;
SAPISID=kabRBO-uT0ebDcfc/A2byDX--FwN649tHw
Host: www.google.com
Connection: Keep-Alive
Accept-Language: ko-KR
User-Agent: Mozilla/5.0 (compatible; MSIE 9.0; Windows NT 6.1; Trident/5.0)
```

HTTP Response Message Format




```
HTTP/1.1 200 OK
Date: Mon, 30 Sep 2013 05:06:42 GMT
Expires: -1
Cache-Control: private, max-age=0
Content-Type: text/html; charset=UTF-8
Set-Cookie:
PREF=ID=882fd677c2891d8f:U=a2f97ca74a99823d:FF=4:LD=ko:TM=1380265256
expires=Wed, 30-Sep-2015 05:06:42 GMT; path=/; domain=.google.co.kr
Server: gws
X-XSS-Protection: 1; mode=block
X-Frame-Options: SAMEORIGIN
Alternate-Protocol: 443:quic
Content-Length: 112239

<!doctype html><html itemscope="" itemtype="http://schema.org/WebPage"> <head>
content="IE=edge"> <meta content="text/html; charset=UTF-8" http-equiv="content-type">
content="/images/google_favicon_128.png" itemprop="image"> <title>Google</title> <script>
window.google={kEI:"4gZJUrS8HMPUigfy5YGYDQ",getEI:function(a){for(var
```

<<HTTP Response Code>>

실행 결과 코드	내용	설명
100번대	정보 전송	HTTP 1.0까지는 계열에 대한 정의가 이루어지지 않았기 때문에 실험 용도 외에는 100번대 서버 측의 응답이 없다.
200번대	성공	클라이언트의 요구가 성공적으로 수신 및 처리되었음을 의미한다.
300번대	리다이렉션	해당 요구 사항을 처리하기 위해 사용자 에이전트가 수행해야 할 추가적인 동작이 있음을 의미한다.
400번대	클라이언트 측 에러	클라이언트에 오류가 발생했을 때 사용한다. 예를 들면 클라이언트가 서버에 보내는 요구 메시지를 완전히 처리하지 못한 경우 등이다.
500번대	서버 측 에러	서버 자체에서 발생한 오류 상황이나 요구 사항을 제대로 처리할 수 없을 때 사용한다.